מכון טכנולוגי חולון
Holon Institute of Technology

הפקולטה להנדסת חשמל ואלקטרוניקה

ספר פרויקט – תואר ראשון

פיתוח מערכת לזיהוי תווים מוסיקליים

מאת:

נדב אושרוב

מנחה: ד"ר נונל טירר

תאריך: 06/06/2025

מכון טכנולוגי חולון
Holon Institute of Technology

Faculty of Electrical Engineering and Electronics

Bachelor's Degree Project Book

Development of a System for Musical Note Recognition

BY:

Nadav Osherov

Supervisor: Dr. Nonel Tirer

Date:06/06/2025

תקצירים

תקציר

הפרויקט הוא מערכת לזיהוי תווים מוסיקליים המשתמשת בחומרה ותוכנה משולבת.

המערכת כוללת בין היתר: מצלמה, FPGA, ארדואינו וממשק משתמש גרפי ב Python.

המצלמה לוכדת תמונה של תווים על תווים מוסיקליים, ה FPGA-מעבד את התמונה ושולח אותה למחשב, שם ממשק ה GUI-מזהה את התו המוסיקלי בעזרת אלגוריתם זיהוי קווים אדומים ושחורים. לאחר מכן, הארדואינו מפיק את הצליל המתאים דרך באזר פסיבי. הפרויקט משלב רכיבי חומרה ותוכנה לפתרון מעשי לזיהוי תווים בזמן אמת.

Summary

The project is a system for musical note recognition that uses integrated hardware and software. The system includes, among other components, a camera, FPGA, Arduino, and a Python-based graphical user interface. The camera captures an image of notes on a musical staff, the FPGA processes the image and sends it to a computer, where the GUI identifies the musical note using an algorithm that detects red and black lines. Then, the Arduino generates the corresponding sound through a passive buzzer. The project combines hardware and software components to provide a practical solution for real-time note recognition.

EXECUTIVE SUMMARY

Abstract

Music is an integral part of human culture and graphic records of it can be found since the early Middle Ages. Almost a millennium ago, the most common method of graphically recording notes that has survived to this day was developed, the "Staff" method, in which notes are written on five horizontal lines.

Despite the great affinity people have for music, the vast majority of them do not know how to read musical notes, let alone know how they sound from reading alone (which is a skill that most musicians do not have neither).

In recent years, there have been numerous attempts to develop systems that can play notes by photographing them. Most of them are applications rather than stand-alone systems and use machine learning models to perform the OMR, although the models do not work optimally at recognizing musical notes at the same level as they do on other characters [1].

Target

The goal of the project is to develop a system that includes a camera and a passive buzzer, which will enable capturing musical notes from a sheet and playing them using the buzzer.

Project Process:

This project integrates FPGA, Arduino, a camera module, and a Python-based GUI to identify and play musical notes. The system captures a photo of a musical note (with a red line below it), processes the image to detect the note, and then plays the corresponding sound through a passive buzzer.

The project involves four main development parts:

1. Writing code for the FPGA
2. Writing code for the Arduino
3. Developing the Python GUI
4. Creating the note recognition mechanism in Python

1. FPGA Code Development

The FPGA (implemented using a development board with MicroBlaze soft processor) handles the following tasks:

OV7670 Camera Configuration:
The camera is initialized using the SCCB protocol (similar to I2C). A dedicated module with a configuration ROM sends the necessary settings.

Image Capture:
The cam_capture module receives pixel data from the camera and stores it into BRAM (Block RAM). Only a 170×200 pixel region containing the note is captured.

VGA Display Output:
A VGA module reads from BRAM and displays the image on a screen using standard 640×480 VGA timing. A red reference line is drawn to help in note identification.

UART Communication with PC:
MicroBlaze sends the stored image over UART to the PC in text format (as pixel values). The data is processed later by the GUI.

Additional Components:

Clock Generator: Produces 25 MHz and 24 MHz clocks for VGA and camera.

<u>Debouncer:</u> Stabilizes push-button input.

<u>Rising Edge Detector:</u> Ensures single BRAM read per press.

<u>Seven-Segment Display:</u> Shows photo index and total requests.

<u>LED Indicators:</u> Reflect system status (controlled by C code).

2. Arduino Code Development

The Arduino receives the detected musical note via UART and plays the appropriate sound using a passive buzzer connected to pin 8. The code uses a frequency mapping of notes (e.g., DO, RE, MI...) to output PWM signals with the correct tone and duration.

3. GUI Development (Python)

The GUI, built with tkinter, allows the user to:

Initiate the photo-taking process.

Visualize the received image.

Trigger note recognition.

Send the detected note to the Arduino.

It acts as the central interface that connects the FPGA's image output with the Arduino's sound output.

4. Note Recognition Mechanism in Python

Once the image (with the red line) is received from the FPGA in text format, the recognition process includes several steps:

Step 1: Image Reconstruction

The image is reconstructed from the text file using cv2 and numpy.

Step 2: Binarization and Preprocessing

The image is converted to binary (black and white).

A Gaussian blur is applied to reduce noise and improve line detection accuracy.

Step 3: Detecting the Staff Lines

A function is used to find the five black lines of the musical staff:

```
staff_lines = find_and_filter_staff_lines(binary, max_lines=5)
```

Uses np.sum(binary_image, axis=1) to project image brightness along rows.

A threshold is set at 70% of the maximum intensity to identify potential horizontal lines:

```
threshold = 0.7 * np.max(projection)

potential_lines = np.where(projection > threshold)[0]
```

The code then merges lines that appear in continuous rows (i.e., thick lines), treating them as one.

Further filtering is done to eliminate false positives: if two lines are closer than 60% of the average distance between staff lines, one of them is removed.

Only the first five valid staff lines are stored for further comparison.

Step 4: Red Line Detection

The RGB image is loaded using:

image = cv2.imread(uploaded_filename)

The image is converted to HSV (Hue, Saturation, Value) color space, which is more effective for color-based segmentation.

The red hue is found in two ranges: 0–10 and 170–180. Additional constraints on saturation (70–255) and value (50–255) ensure only strong reds are captured.

A binary mask is created where the red line appears as white, and everything else is black.

The vertical position (row) of the red line is then calculated.

Step 5: Note Identification

Now with:

An array of five staff line heights (from black line detection)

The vertical height of the red line

The system identifies the note by comparing the red line’s position relative to the black lines.

A predefined array of note names (from highest to lowest) is declared.

The red line's location is compared with each pair of adjacent black lines.

If it falls between two lines, the note is interpolated based on its relative distance.

Slight deviations are accounted for to correct minor noise or imperfect placement.

To sum up, this project demonstrates an end-to-end system combining hardware and software:

Hardware (FPGA + Camera + VGA + UART + BRAM)

Software (Python + OpenCV + GUI + UART)

Microcontroller (Arduino + Buzzer)

The modular design and well-defined stages make it a strong educational example of real-time embedded image processing.

Project Result Summary

In our project, we implemented a musical note recognition system based on simple image processing algorithms. We achieved nearly 100% accuracy in identifying basic white musical notes using a red reference line for detection.

While the recognition was highly successful within our limited set of test images, the project faced several constraints, mainly due to the low-quality camera and the slow UART communication between the FPGA and PC, which limited both data transfer speed and the number of samples we could test.

Some of the key limitations observed:

1. The system cannot currently recognize sharps, flats, or natural signs.
2. It does not identify note durations (whole, half, quarter).
3. Initial attempts using ellipse detection yielded less reliable results on low-resolution images, leading us to switch to the red-line method, which proved more robust under our conditions.

Overall, the system demonstrates that simple tools can achieve effective results in note recognition, and it provides a solid foundation for future enhancements such as duration detection and full music sheet interpretation.

תוכן עניינים

1 מבוא

1.1 רקע כללי 13

1.2 מטרת הפרויקט 14

1.3 דרישות הפרויקט 14

2 רקע תיאורטי 14

2.1 FPGA 15

2.1.1 מיקרובלייז 15

2.2 מיקרובקרים 15

2.2.1 ארדואינו אונו 15

2.2.2 ראספברריי פאי 16

2.3 פרוטוקולי תקשורת 16

2.3.1 פרוטוקול UART 16

2.3.2 פרוטוקול SPI 17

2.3.3 פרוטוקול I2C 17

2.4 ראייה ממוחשבת 18

2.4.1 מודלים לקידוד תמונה 18

2.4.2 VGA 19

2.4.3 פילטרים לעיבוד תמונה 19

2.4.4 OpenCV 21

2.5 בסיס תאורטי במוזיקה 22

2.5.1 צליל כתופעה פיזיקלית 22

2.5.2 באזרים 23

2.5.3 רקע בסיסי בקריאת תווים 23

3 פיתוח מערכת לזיהוי תויים מוסיקליים 24

3.1 הצגת המערכת 24

3.2 פתרונות אלטרנטיביים 26

3.2.1 שימוש במודלים של למידה עמוקה 26

3.2.2 יצירת כל המערכת על מחשב בודד 26

3.2.3 שימוש באלגוריתמים של ראייה ממוחשבת בלבד 26

3.3 שאלת המחקר 27

4 מהלך העבודה 28

4.1 מתודולוגיה 28

4.2 מימוש 28

4.2.1 ערכת פיתוח מבוססת FPGA 30

4.2.1.1 דיאגרמת בלוקים 30

4.2.1.2 תיאור שעונים 30

4.2.1.3 תיאור חלקי ה FPGA 31

4.2.1.3.1 מודל המצלמה OV7670 31

4.2.1.3.1.1 cam_init 31

4.2.1.3.1.2 cam_capture 31
4.2.1.3.2 debouncer 31
4.2.1.3.3 VGA_TOP 32
4.2.1.3.4 design_1 32
4.2.1.3.5 simple_seven_segment 33
4.2.1.3.6 Signal_generator 33
4.2.1.3.7 BRAM 33
4.2.1.3.8 Rising_edge_detector 33
4.2.1.3.9 Clock_gen 34
4.2.1.3.10 ILA 34
4.2.1.3.11 תיאור לחצנים, LEDS ו SW בשימוש בערכת הפיתוח פרויקט 34
4.2.2 GUI מבוסס Python 35
4.2.2.1 מסך 1- פתיחה 35
4.2.2.2 מסך 2- Power UP 36
4.2.2.3 מסך 3- שאלה למשתמש 37
4.2.2.4 מסך 4- כמות התמונות 38
4.2.2.5 מסך 5- הוראות לצילום התמונה 39
4.2.2.6 מסך 6- הגדרות 39
4.2.2.7 מסך 7- הרצת ויטיס 40
4.2.2.8 מסך 8- העלאת קובץ טקסט 41
4.2.2.9 מסך 9- מסך הזיהוי 42
4.2.2.10 מסך 10- מסך החלטה 44
4.2.2.11 מסך 11- מסך ניגון התווים 45
4.2.2.12 מסך 12- מסך פרידה 46
4.2.3 Arduino 47
4.2.4 OV7670 48
4.2.5 תרשים זרימה של הפרויקט 48
5 תוצאות והצעות להמשך מחקר 49
6 מסקנות 57
7 ביבליוגרפיה 59

רשימת איורים

איור 1 : הפורטים של ארדואינו אונו 16
איור 2 : תמונה של פרח ב RGB Grayscale ו Binary 18
איור 3 : גרף סינוסואידלי של לחץ כתלות בזמן 22
איור 4 : השוואה גרפית בין תווים שונים 22
איור 5 : רצף תווים של אוקטבה רביעית..... 23
איור 6 : צורות שונות של התווים לפי משקלים..... 23
איור 7 : תרשים בלוקים 24
איור 8 : מבט על המערכת..... 29
איור 9 : דיאגרמת בלוקים 30
איור 10 : מסך 1- פתיחה 35
איור 11 : מסך 2- Power UP..... 36
איור 12 : מסך 3- שאלה למשתמש..... 37
איור 13 : מסך 4- כמות התמונות 38
איור 14 : מסך 4- כמות התמונות לאחר לחיצה על Enter 38
איור 15 : מסך 5- הוראות לצילום התמונה 39
איור 16 : מסך 6- הגדרות..... 39
איור 17 : מסך 7- הרצת ויטיס 40
איור 18 : מסך 8- העלאת קובץ טקסט 41
איור 19 : מסך 8- לאחר לחיצה על Generate Image..... 41
איור 20 : מסך 9- מסך זיהוי 42
איור 21 : מסך 9- לאחר לחיצה על Detect music char..... 42
איור 22 : מסך 10- מסך החלטה..... 44
איור 23 : מסך 11- מסך ניגון התווים 45
איור 24 : מסך 11- מסך ניגון התווים לאחר העלאת הקובץ 45
איור 25 : מסך 12- מסך פרידה..... 46
איור 26 : תרשים זרימה של הפרויקט 48
איור 27 : רצף של תווים 49
איור 28 : רצף של תווים במשקלים שונים 50
איור 29 : קובץ טקסט של התווים שצריכים להתנגן..... 52
איור 30 : רצף של תווים עם סימנים שונים 52
איור 31 : קובץ טקסט של התווים 54
איור 32 : רצף התווים הראשונים של יונתן הקטן 54
איור 33 : קובץ טקסט של התווים הראשונים של יונתן הקטן 56

רשימת טבלאות

טבלה 1 : תיאור לחצנים, LEDS ו- SW בשימוש בערכת הפיתוח פרויקט 34
טבלה 2 : תוצאות של מספר דגימות של תמונות של תווים..... 49
טבלה 3 : תוצאות של רצף תווים עם משקלים שונים 51
טבלה 4 : תוצאות של רצף תווים עם סימנים שונים 54
טבלה 5 : תוצאות של רצף התווים הראשונים של יונתן הקטן 55

רשימת קיצורים

OMR – Optical Music Recognition

FPGA – Field-Programmable Gate Array

IP- Instruction Pointer

IC- Integrated Circuit

CPU- Central Processing Unit

I/O- Input/Output

RISC- Reduced Instruction Set Computing

PWM- Pulse-Width Modulation

ICSP- In-Circuit Serial Programming

EEPROM- Electronically Erasable Programmable Read-Only Memory

UART- Universal Asynchronous Receiver-Transmitter

SPI – Serial Peripheral interface

SCLK- Serial Clock

MOSI- Master Out Slave In

MISO- Master In Slave Out

SS- Slave Select

I^2C- Inter-Integrated Circuit

SDA- Serial Data

SCL- Serial Clock

RGB- Red Green Blue

VGA- Video Graphics Array

HDMI- High-Definition Multimedia Interface

RAM- Random Access Memory

USB- Universal Serial Bus

PMOD- Peripheral Module Interface

PC- Personal Computer

SCCB – *Serial Camera Control Bus*

ROM – *Read-Only Memory*

LED – *Light Emitting Diode*

GUI – *Graphical User Interface*

ILA-Integereted Logic Analyzer

1 מבוא

1.1 רקע כללי

מוזיקה היא חלק בלתי נפרד מהתרבות האנושית וניתן למצוא תיעודים גרפים שלה עוד מימי הביניים המוקדמים. לפני כמעט מילניום התפתחה השיטה הנפוצה ביותר לרישום תווים שנופצה עד ימינו אנו, היא שיטת ה"חַמְשָׁה" בה התווים כתובים על גבי חמישה קווים אופקיים.

אף על פי הזיקה הרבה שיש לאנשים למוזיקה, רובם המוחלט לא יודע לקרוא תווים מוסיקליים, על אחת כמה וכמה לדעת איך הם נשמעים מקריאה בלבד (שזו מיונות שגם למרבית המוזיקאים אין).

בשנים האחרונות נעשה מספר רב של ניסיונות לפתח מערכות שיידעו לנגן תווים על ידי צילום שלהם. רובן ככולן הן אפליקציות ולא מערכות שעומדות בפני עצמן ומשתמשות במודלים של למידת מכונה על מנת לבצע את ה OMR, אם כי המודלים אינם עובדים בצורה מיטבית על זיהוי תווים מוסיקליים באותה רמה של זיהוי תווים שאינם מוסיקליים [1].

1.2 מטרת הפרויקט

מטרת הפרויקט היא לפתח מערכת הכוללת מצלמה ובאזר פאסיבי שבאמצעותה יהיה ניתן לצלם את התווים השונים מתוך דף התווים לשמוע אותם מתנגנים על ידי הבאזר.

1.3 דרישות הפרויקט

- הכרת העקרונות של כתיבת תווים ברמה בסיסית
- שימוש במצלמה בעזרת FPGA
- למידת והכרת העקרונות של עיבוד תמונה וראייה ממוחשבת
- מימוש התקשורת בין הרכיבים השונים
- תיעוד בספר הפרויקט
- עמידה בלוחות זמנים

2 רקע תאורטי

2.1 FPGA –

רכיב חומרה הניתן לתכנות ומאפשר יצירת מעגלים דיגטליים מותאמים אישית.
ההבדל העיקרי בינוי לבין מעבד הוא שה-FPGA יכול לעבוד בצורה מקבילית כלומר לבצע מספר פעולות במקביל.
ה-FPGA בנוי ממספר חלקים עיקריים:

א. בלוקים לוגיים- יחידות עיבוד בסיסיות שמבצעות פעולות לוגיות וארית‍מטיות.
ב. רשת חיבור- מערכת של קווים שמחברת בין הבלוקים הלוגיים השונים.
ג. יחידות זיכרון- מאפשרות אחסון נתונים.
ד. חיבורי I/O- מאפשרים תקשורת עם רכיבים חיצוניים.

ה-FPGA יכול להגיע לקצבים גבוהים מאוד וניתן למצוא אותו בתעשיות הביטחוניות והרפואיות.

2.1.1 MICROBLAZE-

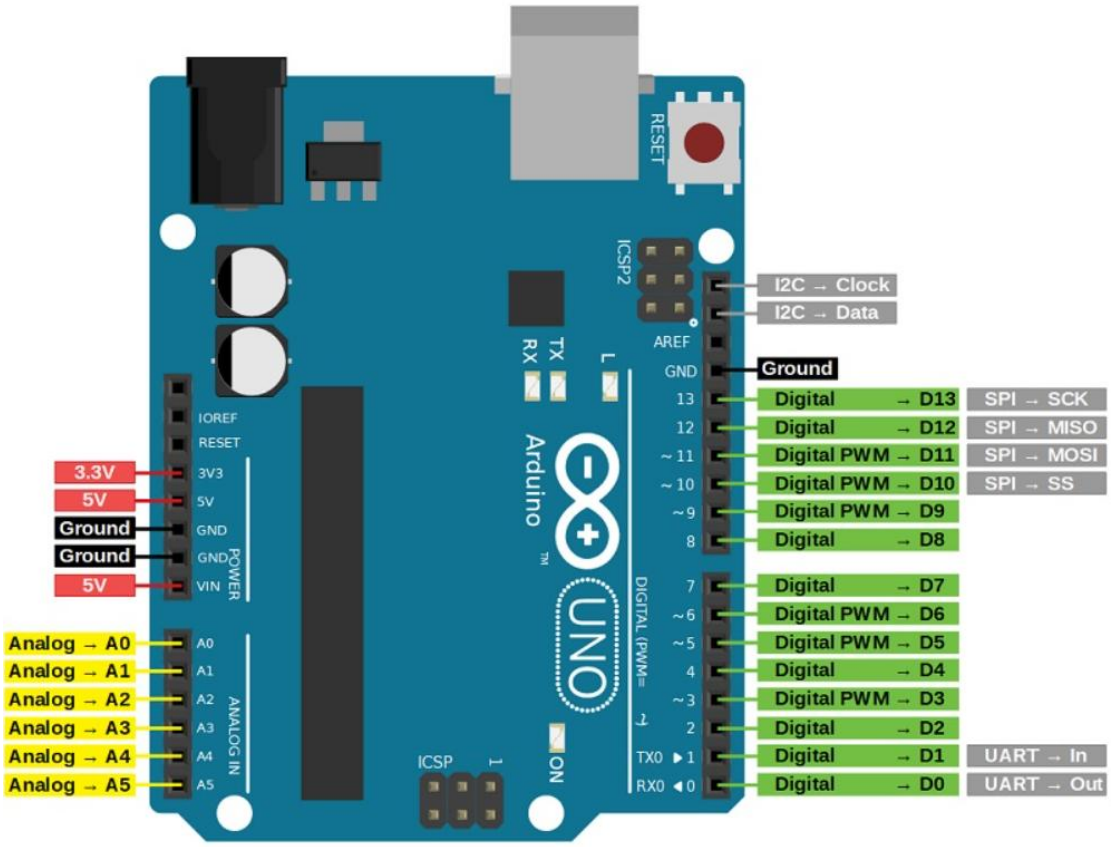
מעבד תוכנתי שנמצא בשימוש בפדטפורמות FPGA של חברת XILINX.
הוא נבנה כמעבד גמיש אשר ניתן להגדירו בקלות.
המיקרובלייז נותן אפשרות למפתחים לכתוב קוד בשפת תכנות ספציפיות (לדוגמה: שפת C) ו"להשתיל" את הקוד הזה בתוך ה-FPGA.
יתרון מרכזי של מיקרובלייז הוא שדרכו ניתן לכתוב ולקרוא מתוך רגיסטרים של IP's מסוימים של חברת XILINX דרך קריאה וכתיבה של מזיכרון בשפת תכנות ספציפיות וללא צורך בכתיבת קוד בשפת חומרה.

2.2 מיקרובקרים –

מיקרובקר הוא מערכת מחשב קטנה ועצמאית המיועדת לבצע משימות ספציפיות.
המיקרובקרים הם בעצם מעגלים משולבים (IC) המשלבים מעבד (CPU), רכיבי זיכרון, רכיבי קלט\פלט (I/O) ועוד.
מיקרו בקרים נמצאים בשימוש נרחב ביישומים שונים כגון רובוטיקה, מכשור רפואי, אוטומצייה תעשייתית וכו'.
למיקרו בקרים יש זיכון שאינו נדיף, מה שמאפשר להם לשמור נתונים גם לאחר שהם מנותקים מהחשמל.

2.2.1 Arduino Uno-

בין מוצרי המיקרו בקרים הנפוצים למשל ניתן למצוא את Arduino Uno. ערכה זו מכילה את המיקרובקר ATmega328 ממשפחת megaAVR של Atmel. מיקרובקר זה מכיל יחידת עיבוד שבנויה בארכיטקטורת הרווארד (בה יש הפרדה בין הזיכרון קוד לזיכרון דאטה) 8-bit RISC (מקבלת בכל פעם דאטה של 8 בתים ומשתמשת בסט הוראות פשוטות ומהירות).
ה Arduino Uno מכיל 14 כניסות\יציאות דיגיטליות (מתוכם 6 יכולות לשמל כ יציאות PWM, 6 כניסות אנלוגיות, כניסת USB ואפשרות לחבר לחשמל ישירות ע"י Power Jack, ICSP (המאפשרים להעלות את התוכנה באופן ישיר מבלי לגעת בחומרה) כפתור אתחול ושעון בתדר 16 MHz. ה ATmega מכיל EEPROM (זיכרון לקריאה בלבד הניתן למחיקה אלקטרונית) בגודל 1kb. בתמונה למטה ניתן לראות את הערכה עם הפורטים השונים שלה.



איור 1 : הפורטים של ארדואינו אונו

2.2.2 Raspberry Pi-

עוד ערכה שנחשבת לעיתים דחופות למיקרובקר היא רסאפברי פאי. דבר זה הוא למעשה טעות מאחר מכיוון שראספברי פאי אינו מיקרו בקר אלא מחשב single-board. לעומת המיקרו בקרים, ראספברי פאי מריץ מערכת הפעלה שלמה (למשל לינוקס) ויכול לעשות פעולות רבות במקביל. ניתן בעזרתו להתחבר לאינטרנט, להריץ סקריפטים של פייתון ועוד. עם זאת עקב הדמיון הרב נהוג לעיתים להתייחס אליו כאל מיקרובקר.

2.3 פרוטוקולי תקשורת-

פרוטוקול תקשורת מתייחס למערכת של כללים, מוסכמות ונהלים המסדירים את חילופי המידע בין שתי ישויות או יותר ברשת תקשורת. הוא מגדיר כיצד נתונים מועברים מתקבלים ומתפרשים, יכול להכיל אימותים ותיקוני שיגאות ומבטיח תקשורת אמינה ויעילה בין מכשירים או מערכות.

2.3.1 פרוטוקול UART-

פרוטוקול תקשורת סדרתית שמטרתו העברת נתונים בין שני מכשירים בצורה פשוטה ויעילה.

בין המכשירים אין חיבור של שעון, כלומר התקשורת היא אסינכרונית.

כל מכשיר שולח את הנתונים בקצב גבוה והקבלה מתבצעת לי הזמן המתואם בין שני הקווים.

בפרוטוקול זה יש שני קווים עיקריים :

TX- משמש לשליחת המידע.

RX- משמש לקבלת המידע.

הפרוטוקול כולל סנכרון כמו start bit לסימון התחלת העברת המידע ו- stop bit לסימון סיום העברת המידע.

ה-UART נמצא בשימוש רחב מכיוון שכמות ההקווים הדרוש לשימוש הוא נמוך מאוד (2 עבור השימוש הפשוט בפרוטוקול) ובשל הפשטות שלו.

2.3.2 פרוטוקול SPI-

פרוטוקול של תקשורת טורית סינכרונית בשימוש רב. פרוטוקול זה הוא דופלקס מלא המאפשר העברת נתונים בין התקן מאסטר למספר התקני עבדים. הממשק מכיל ארבעה קווים.

קו SCLK המשמש לסנכרון העברת הנתונים בין המאסטר לעבדים.

קו MOSI המשמש להעברת הנתונים מהמאסטר לעבד.

קו MISO המשמש להעברת הנתונים מהעבד למאסטר.

קו SS המשמש לבחירת התקן העבד שיתקשר עם המאסטר.

פרוטוקול זה נחשב לפרוטוקול פשוט ומהיר.

2.3.3 פרוטוקול I^2C -

פרוטוקול I^2C הוא פרוטוקול תקשורת סינכרונית טורית. בתקשורת זו משתמשים בשני חוטים בלבד.

קו SDA המשמש להעברת הנתונים מהמאסטר לעבד ולהפך.

קו SCL המשמש לסנכרון התקשורת בין המכשירים.

פרוטוקול I2C משתמש בשיטת התחלה-עצירה של תקשורת.

לפני שהמכשיר הראשי יכול לתקשר עם התקן עבד, עליו לשלוח תחילה אות התחלה כדי לציין את תחילת השידור.

אות ההתחלה הוא מעבר נמוך לגבוה בקו SDA בעוד קו SCL גבוה.

לאחר שליחת אות ההתחלה, המכשיר הראשי שולח את הכתובת של מכשיר העבד איתו הוא רוצה לתקשר.

הכתובת היא בדרך כלל באורך של 7 ביטים, אך ניתן להרחיב אותה ל-10 ביטים עבור מערכות גדולות יותר.

מכשיר העובד עם הכתובת המתאימה יגיב באות אישור.

לאחר שהתקן העבד אישר את ההתקן הראשי, המאסטר יכול לשלוח או לקבל נתונים מהעבד.

המאסטר שולח נתונים על קו SDA בעוד קו SCL גבוה. התקן העבד מקבל את הנתונים ושולח אות אישור למכשיר הראשי.

כאשר המכשיר הראשי סיים לשלוח או לקבל נתונים, הוא שולח אות עצור כדי לציין את סיום השידור.

אות העצירה הוא מעבר גבוה לנמוך בקו SDA בעוד שקו SCL גבוה.

2.4 ראייה ממוחשבת-

ראייה ממוחשבת היא ענף מרכזי של מדעי המחשב העוסק בעיבוד אוטומטי של תמונות, במטרה לחלץ ולפרש מידע חזותי. דבר זה משמש לזיהוי עצמים, סיווג תמונות, זיהוי תמונות וכו'.
אלגוריתמים של ראייה ממוחשבת עושים זאת ע"י שימוש במודלים מתמטיים על התמונות.
התמונות מיוצגות מתמטית ע"י מטריצות המייצגות את ערכי הפיקסלים. ניתן לעשות זאת במודלים שונים, כגון RGB, Grayscale, ייצוג בינארי ועוד.

2.4.1 מודלים לקידוד תמונות-

RGB

מודל צבעים שבו כל צבע נוצר משילוב של שלושה צבעים בסיסיים : אדום,ירוק וכחול.

המודל מבוסס על עקרון של שילוב צבעים כך שכל ערוץ מקבל ערך בטווח מסויים וכך ניתן לייצר צבעים רבים.

נמצא בשימוש נרחב במכשירים אלקטרוניים כמו מסכים ומצלמות דיגיטליות.

מודל זה מאפשר יצירת צבעים רבים תוך השתמשות בשלושה צבעים בלבד.

Grayscale

מודל בו המטריציות הן דו מימדיות וכל ערך מייצג צבע "אפור" שנע בין לבן (לרוב 255) לשחור (לרוב 0)

Binary

תמונות בינאריות מכילות ערכים 0 או 1 וכך מציגות רק את הצבעים שחור ולבן מבלי האפור שבאמצע.

בתמונה הבאה ניתן לראות תמונה של פרח בשלושת המודלים המוזכרים לעיל (כש a זה RGB, b זה Grayscale ו c זה binary)

איור 2 : תמונה של פרח ב RGB Grayscale ו Binary [2]

2.4.2 -VGA

פורמט תצוגה ותקן לקישור בין מחשב למסך.

הוא תומך ברזולוציה של 640 על 480 פיקסלים ומשתמש במספר פורטים למעבר נתוני ה-RGB ומספר פורטים לסנכרון.

למרות שה-VGA הוא תקן ישן יחסית ובתעשייה ישנם ממשקים אחרים כמו DisplayPort ו-HDMI ה-VGA נשאר תקן נפוץ במערכות מסוימות.

2.4.3 פילטרים לעיבוד תמונה-

בשביל להריץ אלגוריתמים של ראייה ממוחשבת צריך תחילה לבצע עיבוד תמונה ברמה המיטבית. חלק חשוב בתהליך זה הוא שימוש במסננים (פילטרים) שישנו את התמונה כך שיהיה ניתן לעבוד איתה ביתר קלות. קיים מספר רב של פילטרים בתחום עיבוד תמונה.

Median Filter

מסנן זה הוא מסנן נפוץ בתחום העיבוד תמונה. הוא משמש בעיקר להורדת רעש "פלפל ומלח". המסנן הוא מסנן לא לינארי והוא עובד כך שהוא לוקח "חלון" עם מספר מסויים של ערכים, מוצא בין ערכים אלו את המספר שהוא החציון ומשאיר רק אותו.

לדוגמה עבור חלון בגדול 3x3 שמקבל את הערכים הבאים

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 8 & 6 \\ 7 & 5 & 9 \end{bmatrix}$$

האלגוריתם תחילה יסדר את המספרים לפי סדר עולה [1,2,3,4,5,6,7,8,9], ואז ייקח את המספר באמצע, זה החציון. החציון הוא 5, לכן הערך במרכז, 8, יוחלף ב 5.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 5 & 9 \end{bmatrix}$$

ככה עושים עבור כל הערכים. עבור הקצוות יש גישות שונות, הנפוצה מבניהן היא להוסיף ריפוד של ערכים שזהים לערכים בקצוות. כך למשל מקבלים בסוף את מהמטריצה הראשונה את המטריצה

$$\begin{bmatrix} 2 & 3 & 3 \\ 4 & 5 & 6 \\ 7 & 7 & 8 \end{bmatrix}$$

Gaussian Filter

טשטוש גאוסיאני הינו טשטוש המשתמש בעקומת גאוס כבסיס ההתמרה. פילטר זה גורם לקווי המתאר בתמונה להיות קלים יותר לזיהוי. הוא גורם לכך שהערכים במרכז מקבלים ערכים גבוהים יותר לפי עקומת גאוס. הפילטר פועל לפי הפונקציה הבאה:

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \qquad (1)$$

כש (x,y) הן הקואורדינטות

σ זה הסטיית תקן. הגדלת סיגמא תיצור טשטוש גדול יותר.

ניקח לדוגמה חלון 3x3. הסטיית תקן תהיה 1. ערכי x,y יראו כך:

$$\begin{bmatrix} (-1,1) & (0,1) & (1,1) \\ (-1,0) & (0,0) & (1,0) \\ (-1,-1) & (0,-1) & (1,-1) \end{bmatrix}$$

לאחר מכן נבצע את החישובים:

$$G(0,0) = \frac{1}{2\pi \cdot 1^2} e^{-\frac{0^2+0^2}{2\cdot 1^2}} = \frac{1}{2\pi} e^0 = \frac{1}{2\pi} \cong 0.159$$

$$G(-1,0) = \frac{1}{2\pi \cdot 1^2} e^{-\frac{(-1)^2+0^2}{2\cdot 1^2}} = \frac{1}{2\pi} e^{-0.5} \cong 0.096$$

וכן הלאה. ניתן לשים לב לסימטריה של הפונקציה וכך לחסוך בחישובים מיותרים. לבסוף נקבל את המטריצה:

$$\begin{bmatrix} 0.058 & 0.096 & 0.058 \\ 0.096 & 0.159 & 0.096 \\ 0.058 & 0.096 & 0.058 \end{bmatrix}$$

השלב הבא הוא ביצוע נורמליזציה. כלומר לחלק כל ערך הסכום של כל הערכים יחדיו.

$$SUM = 0.058 + 0.096 + 0.058 + 0.096 + 0.159 + 0.096 + 0.058 + 0.096 + 0.058 = 0.775$$

כעת מקבלים את המטריצה

$$\begin{bmatrix} 0.075 & 0.124 & 0.075 \\ 0.124 & 0.205 & 0.124 \\ 0.075 & 0.124 & 0.075 \end{bmatrix}$$

בקירוב ניתן להציג את המטריצה הזאת כ

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\2 & 4 & 2\\1 & 2 & 1\end{bmatrix}$$

לצורך הדגמה ניקח שוב את המטריצה הקודמת

$$\begin{bmatrix}1 & 2 & 3\\4 & 8 & 6\\7 & 5 & 9\end{bmatrix}$$

כעת הערך שבמרכז (8) יקבל את הערך הבא :

$$(1x0.075) + (2x0.124) + (3x0.075) + (4x0.124) + (8x0.205) + (6x0.124) + (7x0.075) + (5x0.124) + (9x0.075) = 5.25$$

אם עובדים עם תמונות עם floating point (32-bit או 64-bit) הערך יוכל להישאר כך. אם עובדים עם תמונות 8-bit המספר יעוגל ל 5.

$$\begin{bmatrix}1 & 2 & 3\\4 & 5 & 6\\7 & 5 & 9\end{bmatrix}$$

כך ממשיכים עבור כל הפיקסלים. עבור הקצוות עושים padding בדומה ל Median Filter.

2.4.4 -OpenCV

OpenCV היא חבילת תוכנה שנועדה לעזור למתכנתים לפתח יישומים של ראייה ממוחשבת. התוכנה היא בעלת קוד פתוח. התוכנה נכתבה ב C++ אך עם השנים פותחה ספרייה עבור פייתון (cv2) שמאפשרת להשתמש באלגוריתמים אלה בפייתון וכך מקלה על השימוש בהם.

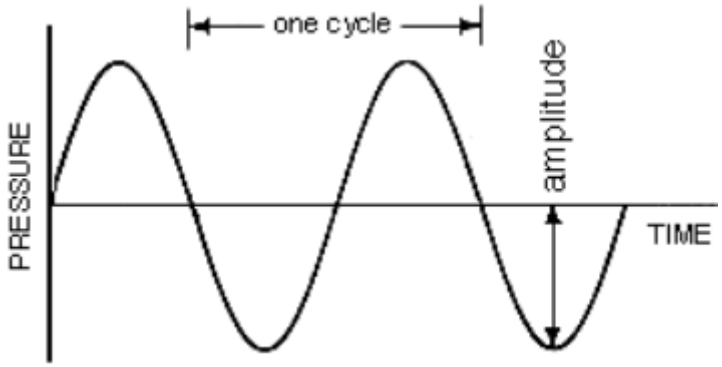
החבילה מכילה פילטרים, זיהוי אובייקטים, הסרת אובייקטים ועוד.

2.5 בסיס תאורטי במוזיקה:

2.5.1 צליל כתופעה פיזיקלית-

התופעה של קול נוצרת כתוצאה של עלייה וירידה של לחץ בתווך מסוים. על תופעה זאת ניתן לחשוב כעל גל המייצג את התפשטות התופעה של שינוי הלחץ בזמן ובמרחב.

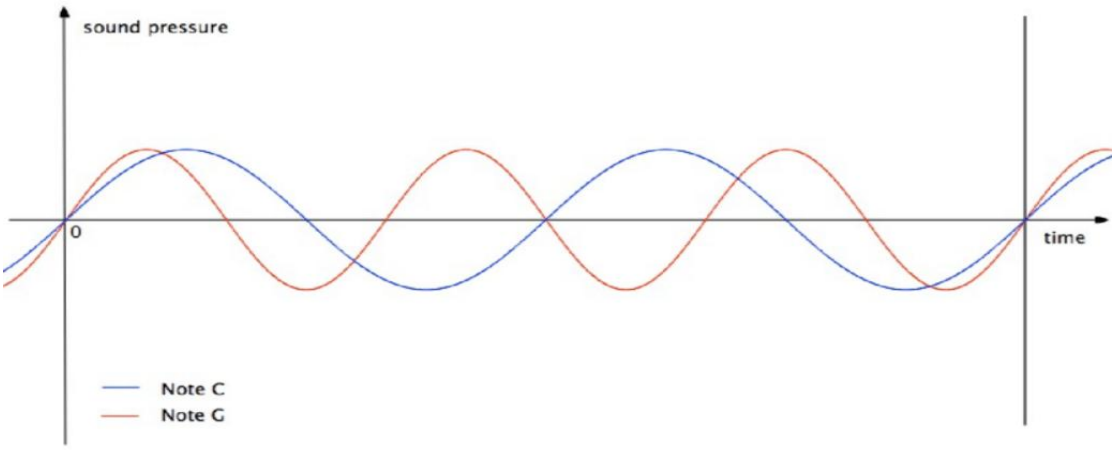
כך למשל באיור הבא ניתן לראות גרף בו הציר האנכי מייצג את הלחץ (מה שמתורגם לאמפליטודה, או לווליום של הצליל, אשר לרוב נמדד בדציבלים) והאופקי את הזמן :



איור 3 : גרף סינוסואידלי של לחץ כתלות בזמן

נהוג לחשוב על צליל "טהור" כגל סינוס, אז ניתן גם להגדיר את התדירות של הגל. במקרה הריאלי הגל הוא אינו גל סינוס, עדיין לפי התמרות פורייה [3] אפשר לפרק אותו למספר (סופי או אינסופי) של גלים סינוסואידלים שונים ולקבוע את תדר הצליל לפי הדומיננטי מבניהם.

כך אפשר למשל לקבוע תווים מוזיקליים. למשל גל בעל 440Hz בעולם המוזיקה ייקרא A4 (לה באוקטבה הרביעית) וכו'. ככל שהתדירות תהיה נמוכה יותר, כך גובה הצליל יהיה נמוך יותר ולהפך. באיור הבא למשל ניתן לראות בצורה איכותית את ההבדל בין התו דו (בכחול) לתו סול שגבוה ממנו (באדום).



איור 4 : השוואה גרפית בין תווים שונים

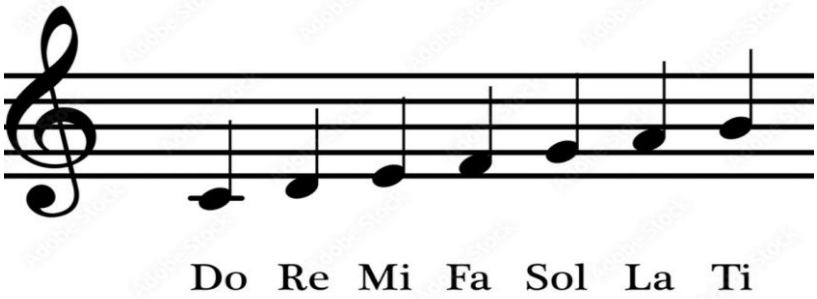
2.5.2 באזרים-

באזר, או בעברית זמזם, הוא רכיב המפיק צליל. באזרים יכולים להיות מכאניים, אלקטרומכניים, או פיאזואלקטרים. פיאזואלקטריות היא תכונה של גבישים קרמיים מסוימים ליצור מתח חשמלי כתוצאה מלחץ מכני ולהפך.

עוד הבחנה חשובה היא ההבחנה בין באזרים אקטיביים לבאזרים פאסיביים. באזרים אקטיביים מכילים מעגל פנימי שמפיק צליל קבוע כאשר הוא מקבל זרם חשמלי. באזרים פאסיביים לעומת זאת יכולים להפיק תדרים שונים לפי המתח שמופעל עליהם.

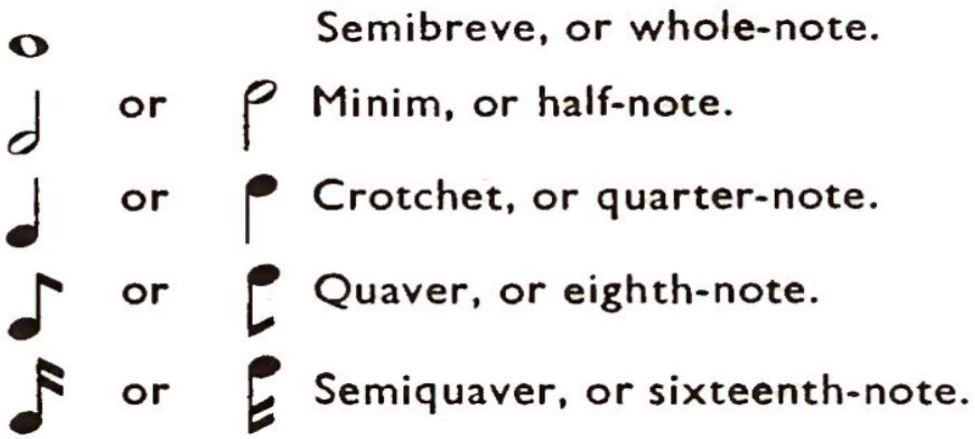
2.5.3 רקע בסיסי בקריאת תווים-

על מנת להבין הפרויקט נדרש ידע בסיסי בקריאת תווים מוזיקליים. כאמור השיטה שמשתמשים בה היום היא שיטת החמשה, בה יש חמישה קווים ואליפסה עליהם\ביניהם. האליפסה מייצגת את גובה הצליל. ככל שהאליפסה נמצאת גבוה יותר כך גם התו נמצא גבוה יותר. האליפסה יכולה להיות מתחת לקו, על הקו או מעליו, כל עלייה אחת כזאת מייצגת עליה בתו אחד. בתמונה הבאה ניתן לראות את רצף התווים דו, רה, מי, פה, סול, לה, סי, דו.



איור 5 : רצף תווים של אוקטבה רביעית

הדבר הבסיסי השני שצריך לדעת בקריאת תווים הוא זיהוי המשקלים. המשקל של התו אומר כמה זמן הוא נמשך (זמן זה הוא היחס בינו לבין אורך התיבה, אך אין צורך להתעמק בזה כעת). עיגול חלול בלי קו הוא שלם, אחריו עיגול חלול עם קו שמסמל חצי, אחריו עיגול מלא עם קו שמסמן רבע, אם מוסיפים עוד קו אחד הצידה זה שמינית. ניתן לראות את התווים באיור הבא.

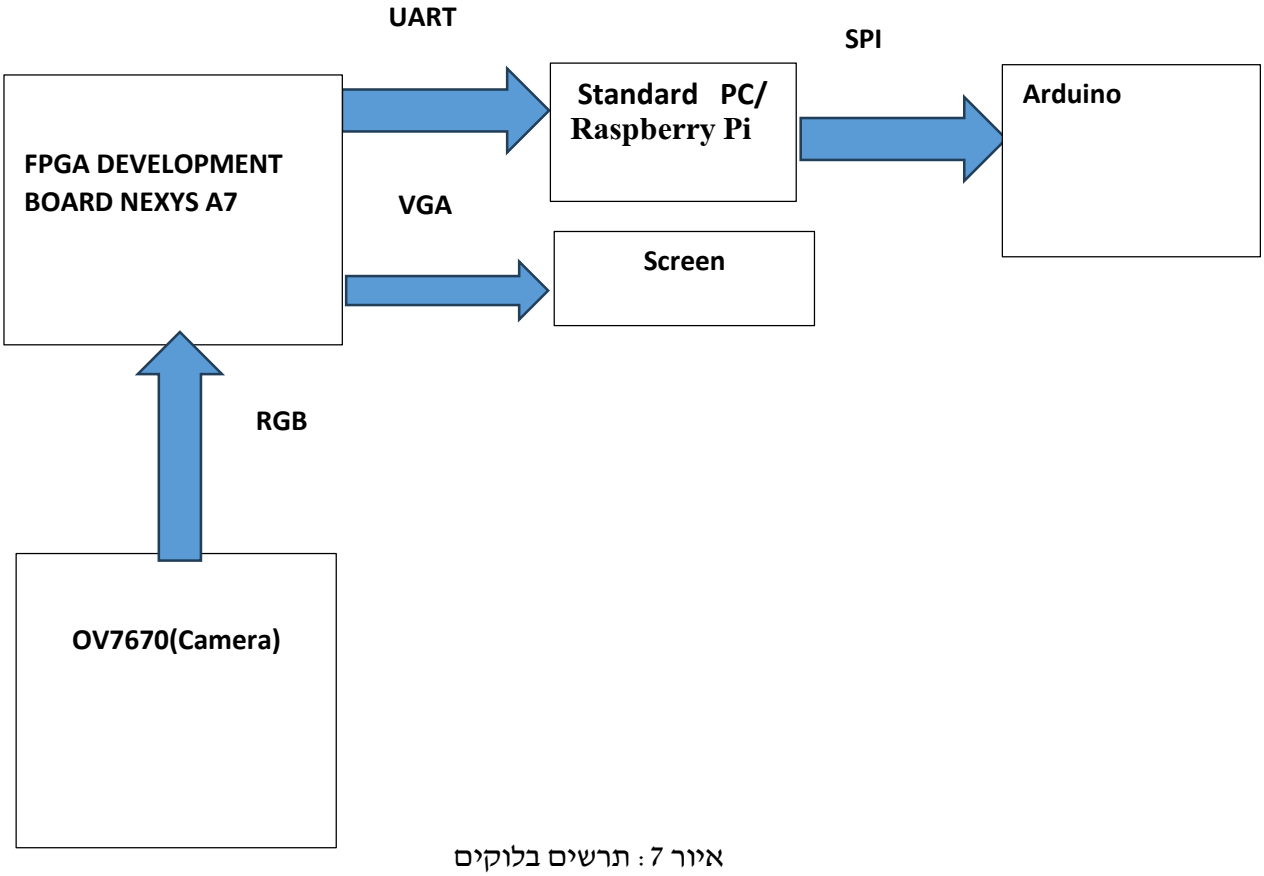


איור 6 : צורות שונות של התווים לפי משקלים

3 פיתוח מערכת לזיהוי תווים מוזיקליים:

3.1 הצגת המערכת-

תרשים בלוקים:



איור 7: תרשים בלוקים

הסבר על הבלוקים:

– OV7670(Camera)

מודל מצלמה בסיס וקטן.

המודל מצויד בחיישן תמונה מסוג CIS ומספק תמונות בחדות גבוהה יחסית למצלמות אחרות בגודל דומה.

פועל על פי פרוטוקול RGB המצלמה מצוידת בפרוטוקול I2C לשליטה בהגדרות המצלמה.

בפרויקט שלנו המצלמה מחוברת דרך פרוטוקול RGB לערכת הפיתוח הכוללת FPGA.

- FPGA DEVELOPMENT BOARD NEXYS A7 לוח פיתוח מבוסס FPGA של חברת XILINX.

הלוח מציע מגוון רחב של יישום פרויקטים בתחומים רבים.

הלוח כולל בין היתר:

- XILINX ARTIX A7 ה-FPGA עליו מבוססת ערכת הפיתוח.

- זיכרון RAM.
- כניסות ויציאות.
- חיבור USB לשם צריבה ו-DEBUG.

בפרויקט שלנו ערכת הפיתוח משמשת לקבלת המידע מהמצלמה דרך חיבור ה-PMOD בפרוטוקול RGB, שליטה בפעולות המצלמה ושליחת הפיקסלים של התמונה שצולמה לכיוון ה-PC דרך פרוטוקול UART.

Screen - מסך מחשב סטנדרטי שתפקידו להראות על המסך את התמונה המשודרת מהמצלמה.

Standard PC – מחשב סטנדרטי שתפקידו לקבל את התמונה המצולמת על ידי המשתמש ודרך כלים מתקדמים בשפת python לבצע עיבוד לתמונה ולזהות באיזה תו מדובר.

את התו המזוהה המחשב שולח דרך ממשק USB וממיר USB ל-SPI אל הארדואינו.

Arduino – פלטפורמת חומרה פתוחה פשוטה ונגישה במיוחד.

הפלטפורמה כולל לוחות מיקרובקרים שמאפשרים למשתמשים ליצור ולבצע פרויקטים אלקטרוניים בצור קלה ומהירה.

תפקיד הפלטפורמה בפרויקט שלנו הוא לקבל את הנתונים לאחר זיהוי התו בתמונה במחשב הסטנדרטי בפרוטוקול SPI ולפי התו שיתקבל מהמחשב לנגן אותו דרך הבאזר.

תיאור הפתרון המוצע בפרויקט:

תפקידו המרכזי של הפרויקט הינו לעזור בלימודי המוזיקה כך שדף תווים יוכל להישמע בצורה קלה כך שהנגן יוכל להבין איך נשמעת המנגינה שעליו לנגן.

המשתמש ישתמש בממשק המשתמש ושם יובאו הוראות מדויקת מרגע סריקת התווים ועד להשמעתם.

לאחר מסך הפתיחה בממשק על המשתמש לכוון את המצלמה לכיוון התווים שאותם הוא רוצה לנגן.

הוא יצטרך לבחור לכמה תווים הוא מעוניין להאזין ובהתאם לכך יהיה מספר התמונות שעליו לצלם.

לדוגמה: אם ירצה להאזין למנגינה בת 8 תווים עליו לצלם 8 תמונות.

במסך הסטנדרטי תופיע התמונה מהמצלמה ועליה קו אדום.

על המשתמש לכוון את הקו האדום לקצה התחתון של התו שאותו הוא מצלם.

לאחר צילום התו עליו לעקוב אחר ההוראות על מנת לשלוח את התמונה המצולמת אל המחשב שם.

לאחר כל שליחה למחשב, המחשב יבצע באופן אוטומטי את זיהוי התו מתוך התמונה ויאגור את התוצאות במקום המיועד לכך.

כך שוב ושוב עד לקבלה וזיהוי של כל התווים אותם המשתמש מעוניין לסרוק.

לאחר מכן, התווים יישלחו אל ערכת הארדואינו והבאזר שאליו מחוברת ערכת הארדואינו ינגן את התווים שצולמו על ידי המשתמש.

3.2 פתרונות אלטרנטיביים :

3.2.1 שימוש במודלים של למידה עמוקה-

בעידן בו אנו חיים כיום השימוש במודלים של למידה עמוקה נהיה יותר ויותר נפוץ והמודלים נהיים יותר ויותר טובים. בנוסף לכך השימוש בשיטה זו יכול להיות מתאים בעיקר כשהתווים אותם אנו מנסים לזהות כתובים ע"י מחשב בפורמטים די אחידים. עם זאת השימוש במודלים של למידת מכונה לא תמיד תואם את התחום של זיהוי תווים מוזיקליים כמו שכבר צוין קודם לכן [1]. צריך גם לקחת בחשבון ששימוש במודלים אלו יכול להכביד על המכשירים בהם אנו עובדים. הבעיה העיקרית בשימוש ביצירת מודלים של למידת מכונה הוא שנרדש dataset גדול ומתאים לבעיה עליה אנו עובדים. עם זאת לא נראה שקיימים dataset-ים מספיק טובים לבעיה שאנו מנסים לפתור. רובם נועדו לזיהוי של תווים בכתב יד ולא נותנים תוצאות של תווים בודדים אלא של כל העמוד ביחד. דבר זה נועד לבעיה הרבה יותר קשה ולכן גם בתחומם הם לא תמיד מביאים תוצאות מספקות ולשימוש בבעיות כמו שלנו הם פשוט לא תואמים. עם זאת ניתן למצוא dataset-ים שהם יותר תואמים לבעיה שלנו, אך התמונות שם הן הומוגניות מדי, וכמו שכבר ניסינו במהלך הפרויקט אפשר לפתח עליהם מודלים שמגיעים למעל ל 95% על סט המבחן שלנקח מהדאטה סט ועדיין לא לקבל תוצאות טובות בתמונות אחרות. בנוסף חלק גדול מהם נועד לזהות רק את צורת התו (כלומר את המשקל שלו) ולא את התו עצמו (דבר שלמעשה גם מקשה לזיהוי הצורה של התו בגלל השוני בתמונות).

ייתכן והפתרון האידיאלי עבור הבעיה שלנו היה יצירת dataset תואם בעזרת הכלים בהם אנו עובדים, אך בהינתן הזמן, המאשבים ועצם העובדה שמספר הלייבלים בדאטה סט יהיה כה רב וכל לייבל יצטרך כמה מאות או אלפי תמונות פתרון זה אינו ראליסטי אך ייתכן שאם מערכת כזאת הייתה מפותחת עבור חברה או יזמים עם מספיק זמן משאבים פתרון זה היה הפתרון המועדף.

3.2.2 יצירת כל המערכת על מחשב בודד-

פתרון מתבקש אחר לבעיה בה אנו עוסקים הוא עבודה על מחשב בודד. דבר זה יכול להיות אפשרי על מחשב אישי סטנדרטי או אפילו על ראספבריי פאי. ככה ניתן לחסוך את העבודה של התקשורת בין הרכיבים, להוריד מכמות הרעשים וההפרעות, ואולי לעבוד עם מכשיר קומפקטי יותר ולזרז תהליכים. עוד יתרון למשל בשימוש במחשב אישי או פלאפון זה שאין בהכרח צורך במצלמה חיצונית ובאזר. אנחנו בחרנו לא ללכת בדרך זו בעיקר למטרות למידה ולרכישת ניסיון במהלך הפרויקט עם כלים וטכניקות שונות.

3.2.3 שימוש באלגוריתמים של ראייה ממוחשבת בלבד-

השימוש במערכת נעשה כך שהמשתמש מצלם בכל פעם תו אחד בנפרד ויש אפשרות לסמן בקו אדום את האליפסה ואז לשלוח את התמונה. אומנם ניתן לחסוך זאת על ידי שימוש באלגוריתמים לזיהוי אובייקטים, אז היה ניתן לצלם את כל התווים בתמונה אחת, ובאלגוריתם של זיהוי אליפסות. כאן בנו צריכים להזכיר שהציוד איתו אנו עובדים נחשב ליחסית פשוט וישן והמצלמה עצמה שאנחנו מחברים ל FPGA אינה נותנת תוצאות מספיק איכותיות. כך למשל יצרנו קוד המכיל שימוש בפונקציה של זיהוי אליפסות עם המון עיבוד מקדים וקיבלנו תוצאות די מספקות על תמונות שנלקחו מהאינטרנט אך תוצאות עם הרבה יותר שגיאות על תמונות שלקנו מה FPGA עמו אנו עובדים. בעקבות כך הועלה הרעיון של שימוש בקו עזר אותו המשתמש ישים ולאחר שליחת התמונה יהיה צורך לזהות רק את הקווים האופקים, שזה אלגוריתם עם הרבה פחות שגיאות מאלגוריתמים של זיהוי אליפסות, ויהיה ניתן לזהות את מיקום האליפסה לפי זיהוי הקו האדום ביחס לשאר הקווים.

3.3 שאלת המחקר :

כפי שכבר צוין לעיל, מערכות שיודעות לקרוא תווים הן נדירות וקשות ליישום. אנו ניסינו לבחון דרך מקורית לגשת לבעיה בה אנו משתמשים בכלים שונים (FPGA, מיקרובקר ומחשב) ובעזרתם מפרקים את הבעיה לחלקים שונים ומנסים לפתור אותה. בנוסף אנחנו לא השתמשנו בלמידת מכונה, כך שאין חשש לאוברפיטינג של המודל. ניסנו לבדוק איך ניתן לנסות לגשת לבעיה המורכבת הזאת על ידי שימוש בכלים פשוטים יותר ובגישה חדשה ומקורית.

4 מהלך העבודה:

4.1 מתודולוגיה-

בחלק התוצאות בהמשך הספר מפורטים אופן איסוף התוצאות בפרויקט שלנו.

שיטת איסוף התוצאות שבה נעשה שימוש היא בדיקה כמותית של כמה שיותר תווים מסוגים שונים ובדיקת הזיהוי שלהם.

קיימים כמה מודלים לזיהוי תווים העיקריים ביניהם נותנים דיוק של מעל 90% בזיהוי התו.

הפרויקט שלנו התקרב מאוד ל-100% מבחינת זיהוי התווים, אך יש לזכור שבדקנו מספר מאוד מוגבל של תווים.

בין המגבלות המשמעותיות שחווינו בפרויקט זה היה איכות המצלמה.

רצינו לבצע את הפרויקט עם כמה שפחות משאבים ושיהיה שווה לכל כיס, ולכן היה צריך להשתמש במודל מצלמה לא יקר.

בנוסף מפאת חוסר הזמן נאלצנו להשתמש בפרוטוקול UART בין ה-FPGA למחשב שהוא יחסית איטי מה שגרם לזמני המתנה ארוכים לזיהוי של כל תו.

זמן נוסף בפרויקט היה גורם לנו לנסות להשתמש בפרוטוקולים חדשים ומהירים (כמו SPI) לשליחת התמונה מה שהיה יכול לקצר מאוד את זמני הזיהוי עבור כל תו.

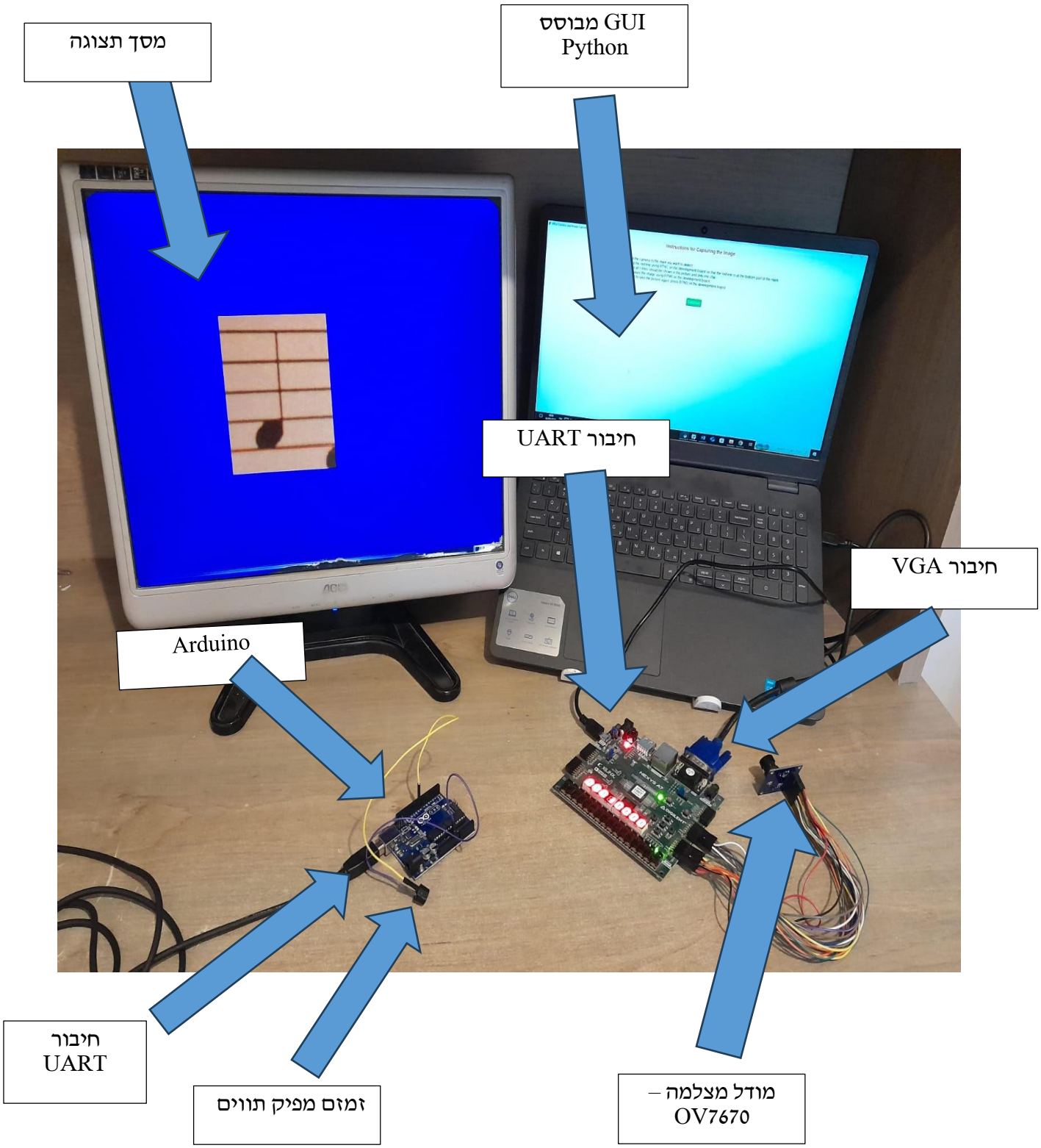
בנוסף, לשם הפשטות החלטנו לוותר על השימוש ב-SPI בפרויקט עבור התקשורת בין המחשב ל-Arduino.

4.2 מימוש-

בפרויקט שלנו יש ארבעה חלקים מרכזיים:

1. ערכת פיתוח מבוססת FPGA- אחראית על הגדרת המצלמה ,צילום, הצגת התמונה במסך ושמירת התמונה בזיכרון הפנימי של ה-FPGA וכן שליחת התמונה דרך פרוטוקול ה-UART אל המחשב.
2. GUI מבוסס פייתון- אחראי על ממשק המשתמש, קבלת התמונה מה-FPGA שליחת התמונה אל הArduino וזיהוי התו שצולם.
3. Arduino- אחראי על קבלת התמונה דרך הUART והפקת הצלילים בהתאם לתמונה.
4. OV7670 מודל מצלמה שאחראי על הפקת התמונה ושליחתו דרך קווי המידע (להוסיף פרוטוקול) אל ערכת הפיתוח.

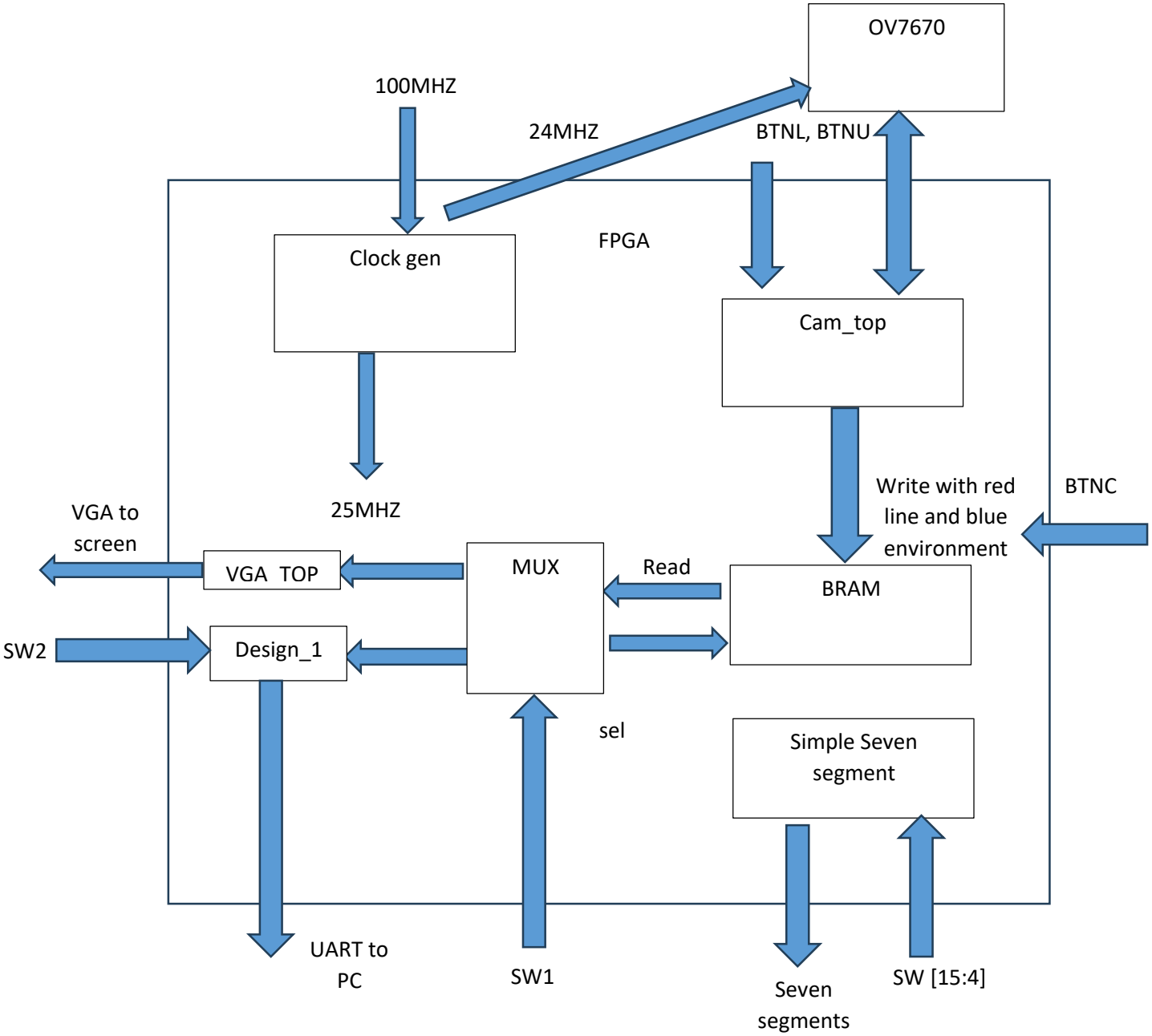
בפרק זה נפרט על כל חלק ונתאר איך הוא בנוי ואיך הוא מתחבר לשאר הרכיבים בפרויקט.



איור 8 : מבט על המערכת

4.2.1 ערכת פיתוח מבוססת FPGA

4.2.1.1 דיאגרמת בלוקים



איור 9 : דיאגרמת בלוקים

4.2.1.2 תיאור שעונים:

בפרויקט קיימים שני clock domains.

הראשון הוא clock domain של 100MHz שאיתו עובד החלק של המצלמה ב-FPGA והכתיבה ל-BRAM.

השני הוא clock domain של 25MHz שאיתו עובד החלק של ה-VGA והקריאה מה-BRAM.

קיים שעון נוסף בפרויקט שהוא של 24MHz שהולך אל מודל המצלמה מחוץ ל-FPGA.

4.2.1.3 תיאור חלקי ה-FPGA:

4.2.1.3.1 מודל המצלמה OV7670 – מודל Verilog שמבוסס על קוד שפורסם בgit.

מודל זה אחראי על אתחול המצלמה מסוג OV7670 באמצעות פרוטוקול SCCB (גרסה של I2C).
גם מודל זה מחולק לכמה חלקים:.

4.2.1.3.1.1 cam_init– מודל שאחראי על שליחת ההגדרות הרצויות לרכיב המצלמה.

חלק זה בנוי משלושה תתי חלקים:

א. זיכרון ROM שתפקידו לאחסן את הגדרות האתחול של המצלמה.
ב. מודל config שתפקידו לשלוף את הערכים מה-ROM ולשלוח למודל ה- SCCB.
ג. מודל ה- SCCB מנהל את השליחה בפועל אל מודל המצלמה דרך קו הנתונים SIO_D וקו השעון SIO_C.

באופן כללי המודל עובד כך שהוא מחכה לאות i_cam_init_start ולאחר מכן המודל מחכה שהמודל SCCB יאותת כשהוא מוכן.

רק לאחר מכן המודל יתחיל לקחת את ההגדרות השמורות ב- ROM ולשלוח אותם למודל המצלמה.

לאחר שכל ההגדרות נשלחו האות o_cam_init_done עולה ל-'1' מה שמעיד על סיום שלב ההגדרה של המצלמה.

בפרויקט שלנו o_cam_init_done מחובר ל- LED1 בערכת הפיתוח והדלקה שלו מעידה על כך ששלב הגדרות המצלמה הסתיים והמצלמה מוכנה לשימוש.

4.2.1.3.1.2 cam_capture – חלק זה מקבל את נתוני הפיקסלים מהמצלמה וממפה אותם לכתובות בזיכרון ה-BRAM שנמצא ב-FPGA.

המודל מתחיל לפעול לאחר סיום האתחול של המצלמה שתואר קודם.

הוא כולל מכונת מצבים המורכבת מארבעה מצבים:

Cam_init- המתנה לאתחול המצלמה וקליטת פריים חדש מהמצלמה.

במודל זה קיימת מכונת מצבים נוספת אשר בוחרת על פי לחצני BTND (מעבר למצב וידאו) ולחצן BTNR (מעבר למצב סטילס-צילום התמונה) את מעברי המצבים במכונת המצבים הראשית שתוארה מוקדם יותר.

4.2.1.3.2 debouncer מודל לסינון רעשים המשמש עבור הלחצנים BTND ו- BTNR.

מטרת מודל זה הוא לוודא שהשינוי בלחצן הוא "אמיתי" ואינו זמני או שנגרם כתוצאה מרעש על הקו.

מודל זה משתמש ב-DELAY של מספר מחזורי שעון (240,000 במקרה שלנו) כאשר אם המצב של הלחצן שונה מהמצב האחרון שהוא שהה בו למספר מחזורי השעון כמו ב-DELAY מכניסים את הערך של הלחצן אל היציאה של המודל וזהו הערך האמיתי של הלחצן שאיתו משתמשים במודל המצלמה.

ה-debouncer במודל זה מוגדר ל-2.4ms.

כלומר שינוי ברמת הקלט תתקבל רק אם המשתמש לחץ על הלחצן למשך 2.4ms ויותר.

4.2.1.3.3 **VGA_TOP** - מודל שנלקח מה- git שתפקידו לקרוא את הנתונים מה-BRAM ולהמיר אותם לנתונים בפרוטוקול VGA.

מודל זה מארגן תזמון תצוגת הווידאו ואת קריאת הפיקסלים מהזיכרון לאורך הצגת התמונה.

מודל זה משתמש במודל אחר שנקרא vga_timing שתפקידו לספק תזמון VGA תקני לפורמט (480x640) שבו אנו משתמשים בפרויקט.

בסטנדרט VGA תקני יש כמה שלבים חוץ משלב בתצוגה שבו נתוני הפיקסל האמיתיים של התמונה.

מודל ה- vga_timing מספק את הקואורדינטות של כל פיקסל כך שניתן לדעת מתי יש אות וידאו תקף ומתי הפיקסל שייך לשוליים או לסנכרון.

לפי מודל זה מודל ה- vga_top יודע מתי יש לשלוח נתוני תמונה ומתי יש לשלוח אותו סנכרון וכך ניתן לדעת מתי ניתן לקרוא נתונים מה-BRAM.

גודל יציאת bus הנתונים של המודל הוא 12 סיביות שמורכבות מארבע סיביות לכל צבע מבין אדום, ירוק וכחול וכך מורכב הפיקסל השלם שנשלח למסך התצוגה ובו גם אנחנו נשתמש לשם זיהוי התו המוסיקלי.

4.2.1.3.4 **design 1** – מודל מבוסס Microblaze שהוא ליבת מעבד שתפקידה בפרויקט הוא לקבל את נתוני הפיקסלים ולשלוח אותם למודל ה-AXI_UART שמקבל את נתוני הפיקסלים דרך פרוטוקול (AXI Memory Mapped) AXI_MM ושולח אותם דרך פרוטוקול UART אל המחשב.

ה-Microblaze הוא בסופו של דבר ליבת מעבד לכן הוספנו לפרויקט קוד C שרץ על ה Microblaze והוא זה שאחראי על העברת הנתונים אל ה-AXI UART דרך דרייברים מובנים שמסופקים עם אפליקציית Vitis שזוהי הסביבה שעליה נכתב קוד ה- C.

בשל שיקולי זמן והאיטיות היחסית של ה-UART החלטנו שהתמונה הנשלחת תהיה קטנה יותר מגודל ה- frame בפרוטוקול ה-VGA שבחרנו.

כלומר במקום לשלוח אל המחשב נתוני פיקסלים של 480x640 פיקסלים אנו שולחים רק נתונים רק של 170x200 וכך אנו חוסכים זמן שליחה יקר.

עוד לפני השליחה הקוד ממתין ל-SW2 בערכת הפיתוח שיהיה ב-'1' ורק לאחר מכן מתחילה הקריאה מה-BRAM ושליחת הנתונים אל ה-UART.ה-MICROBALZE קורא ערכים מה-BRAM שם נשמרת התמונה **אך ורק** מכתובות שם נמצאת התמונה ורק אותן הוא שולח לUART. לאחר סיום קריאה ושליחת הערכים הרלוונטיים דרך ה-UART יש המתנה ל-SW4 שיהיה ב-'1' וזה מסמן על תמונה חדשה שצריכה להישלח ושוב מחכים ל- SW2 שיהיה ב-'1' ושליחה של תמונה נוספת תתחיל לאחר מכן ה MICROBLAZE מקבל את ערכי נתוני החומרה (SW2,SW4) ושולח אל נתוני החומרה דרך IP CORE של חברת Xilinx שנקרא AXI GPIO.ב-IP זה הנתונים מועברים דרך פרוטוקול AXI-MM שממפה את נתוני החומרה אל כתובות AXI שאליהם ניתן לגשת דרך המעבד בקלות.

כלומר, כדי לקרוא את מצב ה-SW נצטרך לקרוא מכתובת מסוימת דרך קוד ה- C וכדי לשלוט על רכיבי חומרה אחרים (לדוגמה LEDS) נצטרך לכתוב לכתובות מסוימות שאליהן מופו ה-LEDS. מיפוי הכתובות מתבצע דרך tab ה-Address Editor ביצירת הblock design ב- Vivado.

ה-MICROBLAZE שלנו שולט גם על כמה LEDS:

LED1 - חיווי שקוד ה- C עלה בצורה תקינה.

LED2- חיווי שמתבצעת שליחה של נתוני התמונה.

LED3- חיווי ששליחת נתוני התמונה אל המחשב הסתיימה.

4.2.1.3.5 **simple_seven_segment**- מודל זה נועד להציג 8 ספרות על 2 מסכי ה seven segment בערכת הפיתוח בסדר הבא:

4 הספרות הימיניות מציגות את הערך הדצימלי של SW[15:5] וזה נועד לחיווי מספר התמונות אותם רוצים לצלם.

4 הספרות השמאליות מציגות את ערך התמונה הנוכחית שמנותחת. ערך זה עולה בכל עלייה של SW4.

במודל זה קיים גם debouncer בשביל SW4 שפועל כמתואר קודם, אך עם DELAY של 1,000,000 מחזורי שעון.

כלומר 10ms.

4.2.1.3.6 **Signal generator-** מודל שנועד לייצור אות שעולה בקפיצות של 10 בכל פעם שיש עלייה בלחצן BTNC בערכת הפיתוח.

כאשר עוברים את 340 שהוא הקו התחתון של התמונה חוזרים חזרה לתחילת התמונה שהיא 140 (גבולות התמונה האמיתיים ללא הרקע הכחול).

לשם זיהוי התו במחשב יש לשלוח תמונה שבה יש קו אדום מתחת לתו המוזיקלי.

בפרויקט שלנו אנו נותנים הוראה למשתמש למקם את הקו האדום שנוצר בתמונת הווידאו מתחת לתו המצולם.

כדי שלמשתמש תהיה גמישות מרבית במיקום הקו יש לו אופציה לשנות את מיקום התו בתוך התמונה שהוא מצלם וזה קורה דרך המודל הזה.

4.2.1.3.7 **BRAM-** זיכרון פנימי בתוך ה-FPGA שנועד לשמור את נתוני הפיקסלים של התמונה המצולמת.

ערכי הנתונים שנשלחים אל ה-BRAM נבחרים דרך MUX שנשלט על ידי ערך הכתובת ב-BRAM.

אם ערך הכתובת הוא באחד בין ערכי התמונה האמיתיים נשלח אל ה-BRAM הערך שהגיע מהמצלמה.

אם ערך הכתובת הוא בערך אחד מהערכים התקניים של הקו האדום נשלח פיקסל בצבע אדום בלבד (כך נוצר קוד אדום אנכי שתואר בסעיף הקודם).

אם ערך הכתובת אינו בין ערכי התמונה האמיתיים (170-340 עבור ערכי ה-x ו- 140-340 עבור ערכי ה-y) נשלח אל ה-BRAM פיקסל בצבע כחול בלבד.

כך מתקבלת תמונה שמורכבת מרקע בצבע כחול, תמונה מהמצלמה וקו אדום(ניתן לראות דוגמה באיור 8).

שינוי זה נעשה כדי לחסוך זמן יקר בשליחת התמונה למחשב.

אנו שולחים אל המחשב רק את התמונה ללא הרקע הכחול לכן אנו שולחים פחות פיקסלים וזמן זיהוי התו הכולל מתקצר.

היעד של הנתונים הנשלחים מה-BRAM נבחרים על ידי MUX שנשלט על ידי SW1.

אם SW1 נמצא ב-'0' הנתונים יישלחו אל המודל ה-VGA והתמונה תופיע על מסך התצוגה.

אם SW1 נמצא ב-'1' הנתונים יישלחו אל מודל ה-design_wrapper ונתוני התמונה (ללא הרקע הכחול אך כולל הקו האדום) יישלחו דרך ה-UART אל המחשב.

4.2.1.3.8 **Rising edge detector-** - מודל שתפקידו לזהות עלייה של signal ודרכו לבצע פקודת read מה- BRAM.

דרך קוד ה-C אנו גורמים לעלייה של סיגנל זה אך אנו רוצים לקרוא כל פעם רק מכתובת אחת לכן יש צורך לזהות עלייה שכזאת ושהסיגנל ביציאה שקורא מה- BRAM יהיה ב-'1' למחזור שעון אחד בלבד.

4.2.1.3.9 -Clock gen IP CORE של חברת Xililnx שנועד לקבל clock בתדר 100MHz ולייצר שני שעונים חדשים בתדרים של: 25MHz עבור ה-VGA DOMAIN ו-24MHz עבור מודל המצלמה החיצוני.

4.2.1.3.10 -ILA IP CORE של חברת Xilinx הנועד ל-debug של סיגנלים מסוימים.

פועל כמעין סקופ דיגיטלי.

4.2.1.3.11 תיאור לחצנים, LEDS ו-SW בשימוש בערכת הפיתוח פרויקט:

שם	תפקיד
BTNC	שולט במיקום הקו האדום בתמונה
BTNR	צילום תמונה
BTNL	התחלת הגדרות מודל המצלמה
BTND	מעבר מצב מסטילס לוידאו
BTNU	אתחול מודל המצלמה
LED0	סיום אתחול מודל המצלמה
LED1	טעינה מוצלחת של ה-FPGA
LED2	התחלת שליחת נתוני התמונה אל המחשב
LED3	סיום שליחת נתוני התמונה אל המחשב
SW1	בחירה בין שליחת נתונים למחשב(1) או ל-VGA(0).
SW2	התחל בשליחת נתוני התמונה למחשב
SW4	העלה את מונה התמונות ב-1
SW15-5	שליטת בתצוגת ה-7 segment

טבלה 1 : תיאור לחצנים, LEDS ו- SW בשימוש בערכת הפיתוח פרויקט

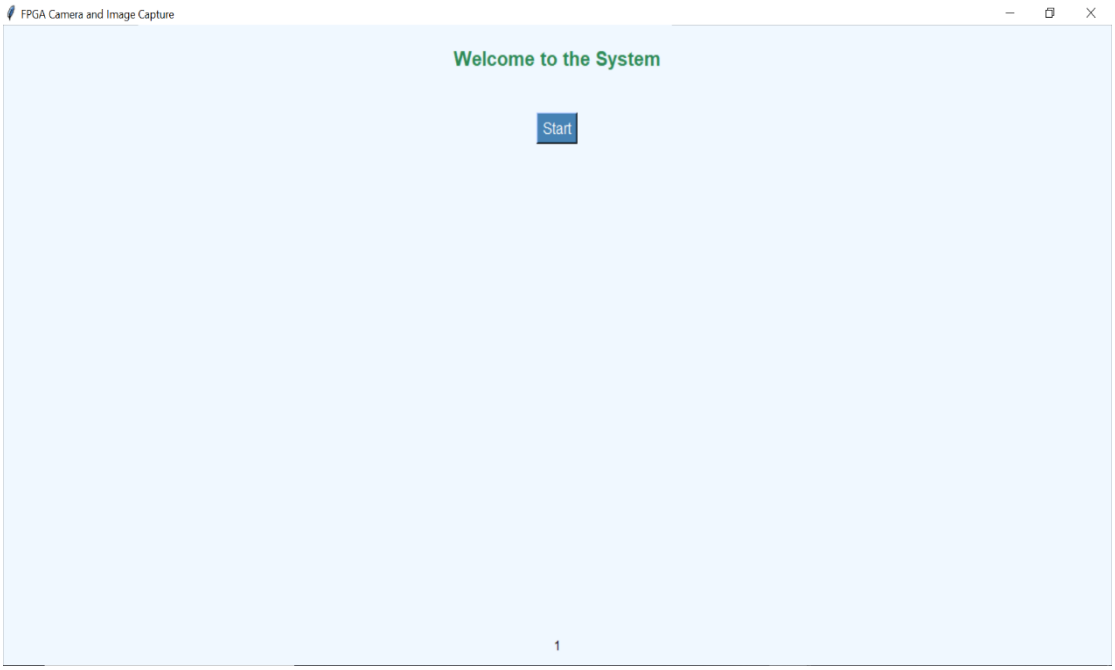
4.2.2 GUI מבוסס Python:

ממשק משתמש שנועד להדריך את המשתמש להשתמש באופן נכון בפרויקט שלנו.

אנו משתמשים בספריית tkinter לשם יצירת ממשק המשתמש.

תיאור המסכים ב-GUI :

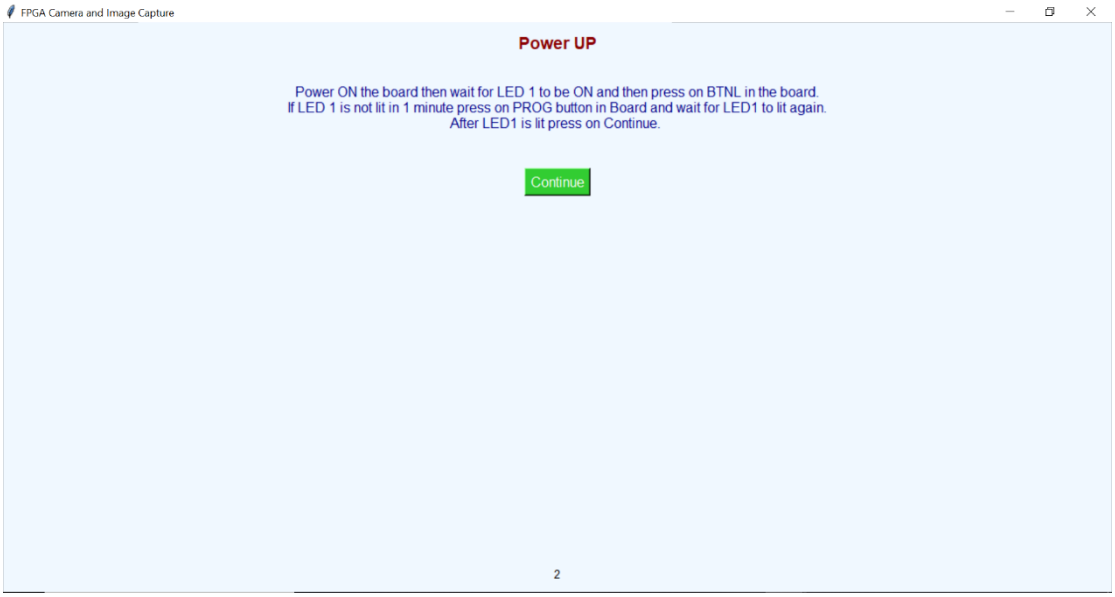
4.2.2.1 **מסך 1-פתיחה-** מסך פתיחה יש ללחוץ Start כדי להתחיל.



איור 10 : מסך 1 - פתיחה

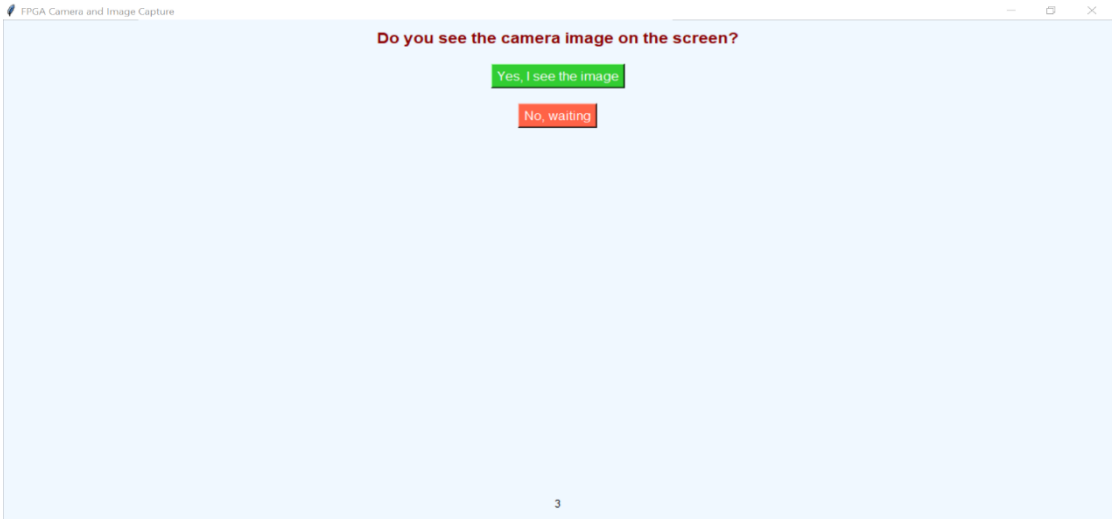
4.2.2.2 **מסך 2 – Power UP-** הדלקת ערכת הפיתוח והמתנה ל- LED 1 שיהיה דלוק (נותן חיווי על טעינה מוצלחת של הערכה) לאחר מכן יש ללחוץ על BTNL אמורה להתחיל את כתיבת ההגדרות למצלמה.

לעיתים ה-FPGA אינו נטען כראוי ואז יש ללחוץ על לחצן PROG בערכת הפיתוח לטעינה חוזרת שלו. ה-bitfile שהוא קובץ הצריבה של ה-FPGA נמצא ברכיב FLASH בערכת הפיתוח כך שלחיצה של PROG גורמת לטעינה שלו מחדש.



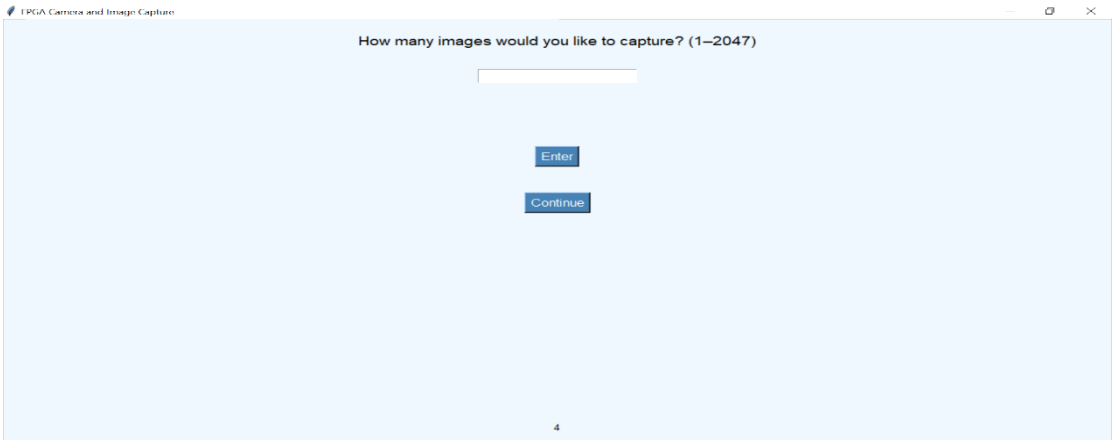
איור 11 : מסך 2 – Power UP

4.2.2.3 **<u>מסך 3- שאלה למשתמש-</u>** שאלה האם רואים את התמונה על המסך. אם כן מתקדמים הלאה.

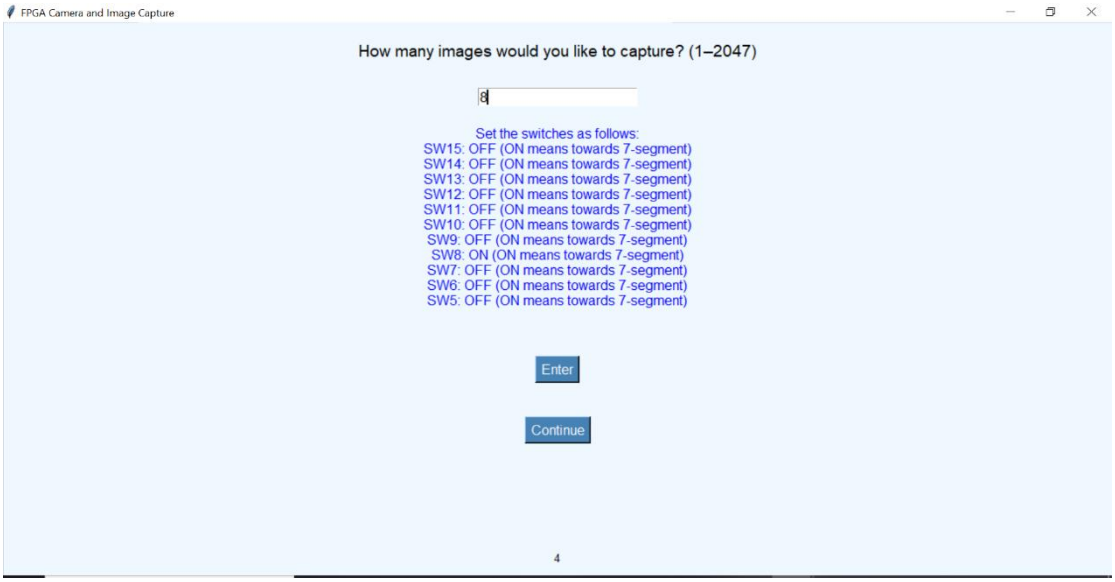


איור 12 : מסך 3 - שאלה למשתמש

4.2.2.4 **מסך 4 -כמות התמונות-** על המשתמש להזין כמה תמונות הוא רוצה לצלם. כלומר כמה תווים הוא רוצה לצלם. לאחר הכנסת הערך ולחיצה על Enter מופיעות הוראות למשתמש איך לשנות את לחצני ה-SW כך שכמות התמונות שאותן רוצים לצלם יופיעו על אחת מתצוגות ה-7segnemt .

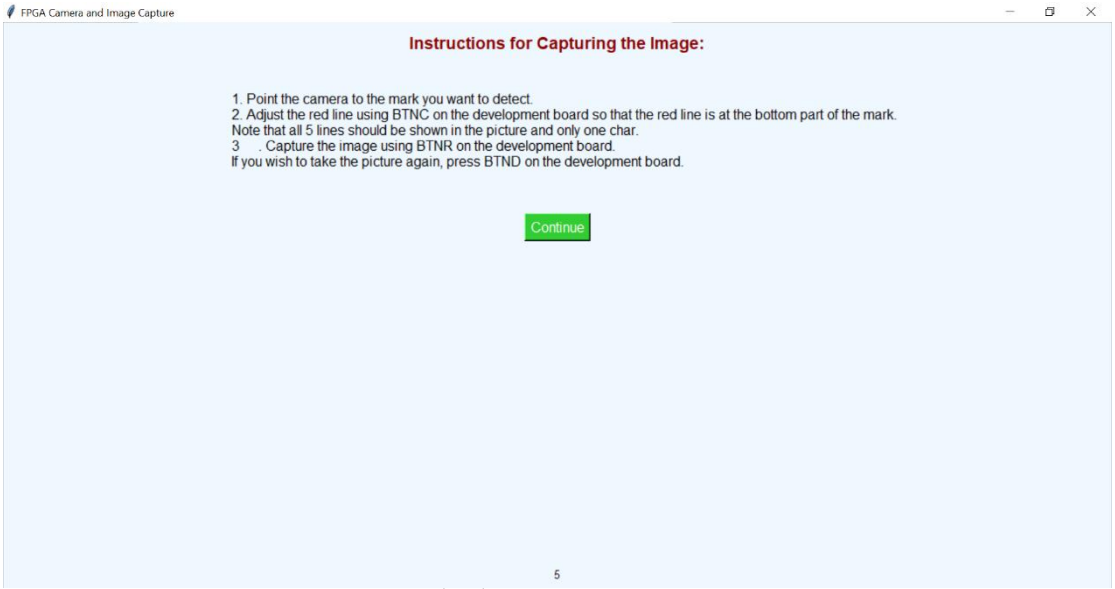


איור 13 : מסך 4 – כמות התמונות



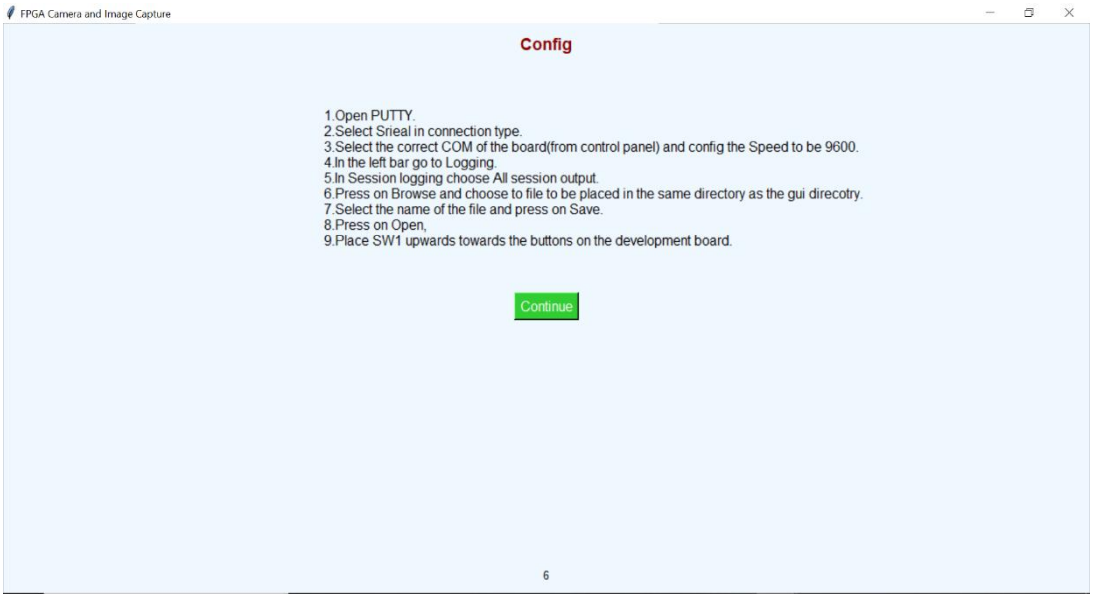
איור 14 : מסך 4 – כמות התמונות לאחר לחיצה על Enter

4.2.2.5 מסך 5-הוראות לצילום התמונה- מסך זה נותן דגשים לצילום התמונה. יש לכוון את הקו האדום דרך לחצן BTNC בערכת הפיתוח כך שהקו האדום יופי מתחת לתו המצולם. לחיצה על BTNR תגרום לצילום התמונה. אם המשתמש לא מרוצה מהתמונה שצילם הוא יכול ללחוץ על BTND כדי לחוזר למצב ווידאו וצילום.



איור 15 : מסך 5 – הוראות לצילום התמונה

4.2.2.6 מסך 6- הגדרות- במסך זה יש הוראות למשתמש כיצד להגדיר את תוכנת ה-PuTTy שמקליטה את נתוני ה-UART שמגיעים אל המחשב מה-FPGA.נתוני ה-UART מכילים את התמונה שהמשתמש צילם. במסך זה על המשתמש להגדיר ל-PuTTy לכתוב את כל הנתונים שמתקבלים בקובץ. קובץ זה ישמש אותנו מאוחר יותר לזיהוי התו לפי נתוני הפיקסלים שמתקבלים. לאחר סיום ההגדרות על המשתמש לשנות את מצב sw1 לכיוון הלחצנים מה שיסמן ל-FPGA לשלוח נתונים אל ה-UART ולא אל ה-VGA.



איור 16 : מסך 6 – הגדרות

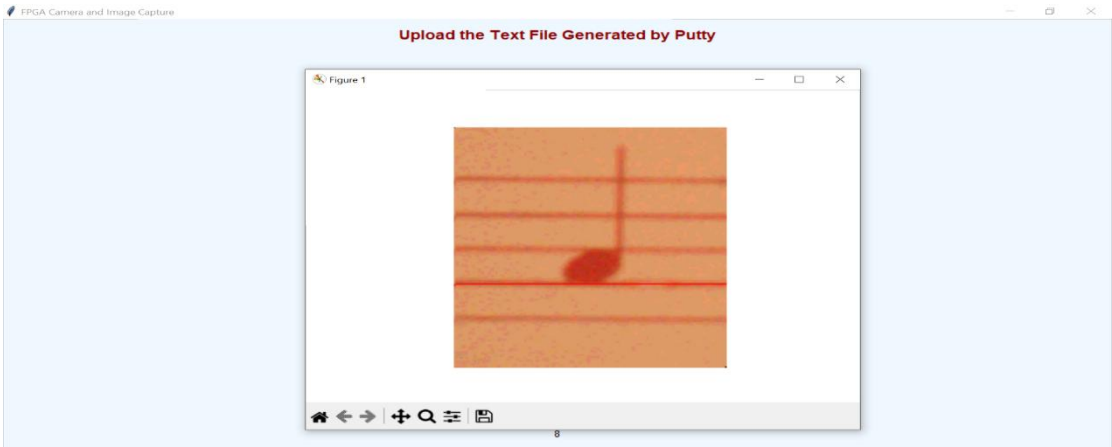
4.2.2.7 מסך 7- הרצת ויטיס- במסך זה על המשתמש לשנות את מצב sw2 לכיוון הלחצנים מה שיסמן ל- Microblaze שבתוך ה-FPGA להתחיל לשלוח את נתוני התמונה אל המחשב דרך ממשק ה-UART. תהליך זה נמשך בערך 15 דקות ולאחר שהתהליך נגמר led3 נדלק ועל המשתמש לסגור את חלון ה-PuTTy. בשלב זה יש לנו קובץ טקסט עם נתוני הפיקסלים של התמונה שהמשתמש צילם.

איור 17 : מסך 7 – הרצת ויטיס

4.2.2.8 **<u>מסך 8- העלאת קובץ טקסט-</u>** במסך זה על המשתמש להעלאות את קובץ הטקסט שנוצר מוקדם יותר.

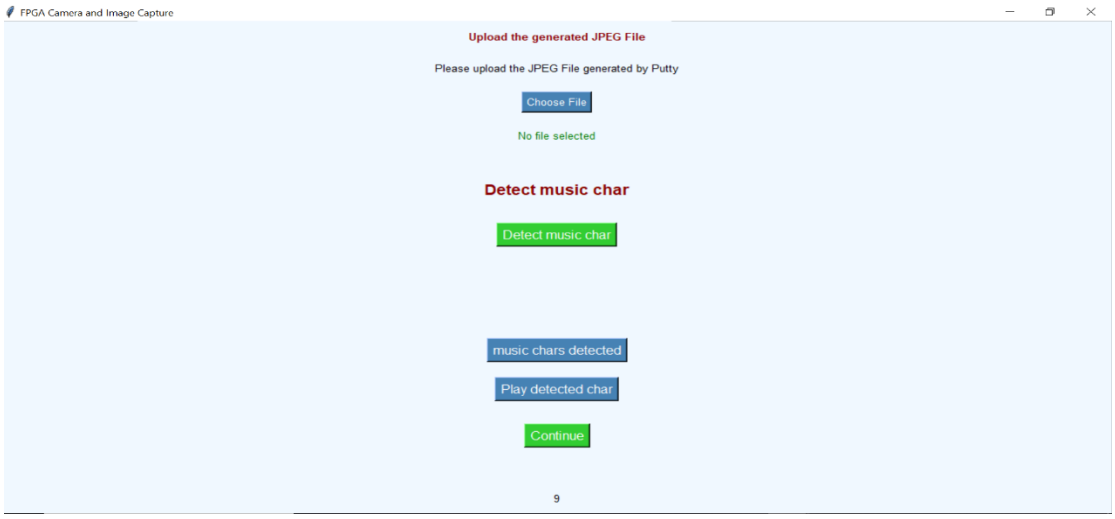
במסך זה על המשתמש ליצור את התמונה שצילם דרך קובץ הטקסט כדי ליצור תמונה בפורמט JPEG שבה נשתמש כדי לזהות את התו במסך הבא.
בנוסף זה נותן ודאות שהתמונה שצילם זאת התמונה שנשלחה אל המחשב.

איור 18 : מסך 8 – העלאת קובץ טקסט

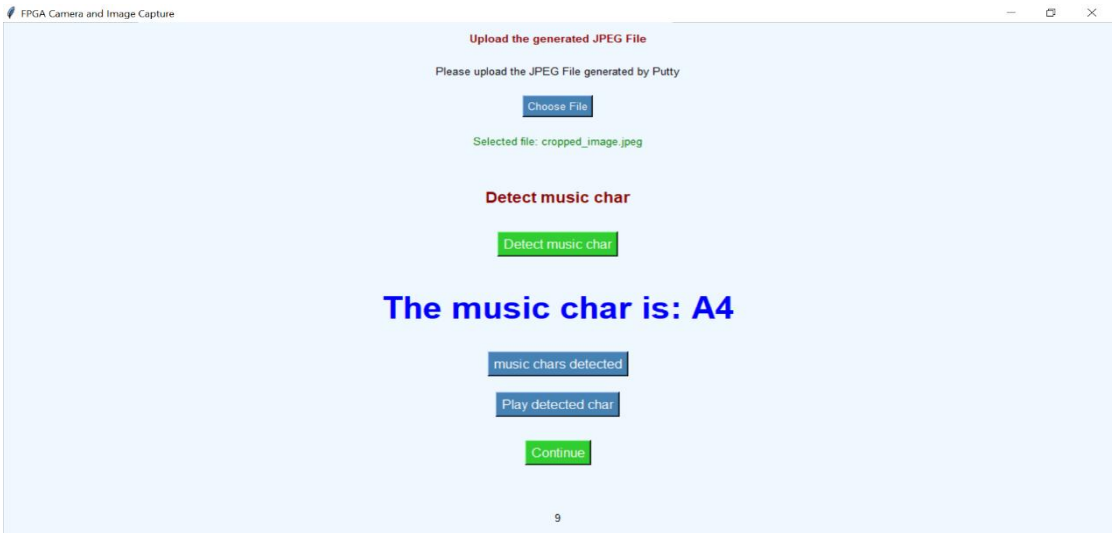


איור 19 : מסך 8 – לאחר לחיצה על Generate Image

4.2.2.9 **<u>מסך 9- מסך הזיהוי-</u>** במסך זה מעלים את קובץ ה-JPEG שנוצר במסך הקודם והכפתור Detect Music char מריץ את קוד הזיהוי וכותב למשתמש איזה תו זוהה. בנוסף למשתמש יש אופציה לקבל את כל התווים שזוהו עד עכשיו דרך כפתור music chars detected ולנגן את התו שהתקבל דרך כפתור Play detected char.



איור 20 : מסך 9– מסך הזיהוי



איור 21 : מסך 9–לאחר לחיצה על Detect music char

מנגנון זיהוי התו:

לאחר קבלת התמונה מה FPGA שמכילה את התו עם הקו האדום כקובץ טקסט ושחזורה נעשה החלק של זיהוי התו. על מנת לבצע פעולה זו כתבנו קוד פייתון שמשתמש בעיקר בספריות cv2 ו numpy.

תחילה שחזרנו את התמונה מהקובץ טקסט. לאחר מכן העברנו את התמונה לפורמט בינארי ועשינו לה טשטוש גאוסייני על מנת להקל על הזיהוי.

בשלב הבא יצרנו פונקציה שתזהה את חמשת הקווים השחורים בתמונה ותשים את הערכים של הגבהים שלהם במערך בערך חמישה איברים. קריאת הפונקציה נראית כך:

```
staff_lines = find_and_filter_staff_lines(binary, max_lines=5)
```

כש binary היא התמונה הבינארית לאחר הטשטוש.

לאחר מכן השתמשנו בפונקציה sum של numpy שסוכמת את כל הערכים בציר שניתן לה. אנו השתמשנו ב axis=1, כלומר ציר אופקי.

```
projection = np.sum(binary_image, axis=1)
```

הצעד הבא הוא זיהוי השורה עם הערך הכי גבוה ודרישה שהקווים יהיו לפחות 70% מהאינטנסיביות של אותה שורה, אותם אנו שומרים כקווים פוטנציאלים.

```
threshold = 0.7 * np.max(projection)
```

```
potential_lines = np.where(projection > threshold)[0]
```

החלק הבא בקוד נועד לשלב בין קווים שנמצאים ברצף אחד אחרי השני, שהרי אם הם נמצאים ברצף מבחינת השורות של התמונה ניתן להסיק שמדובר בקו אחד עבה שתופס כמה שורות וצריך להתייחס אליו כאל קו אחד.

דבר דומה נעשה גם לאחר מכן כשאנו למעשה מחשבים את המרחק הממוצע בין שורות ומורידים שורות אם הן נמצאות בפחות מ 60% מהמרחק הממוצע, כך אנו נמנעים מזיהויים שגויים. בנוסף זה יכול לעזור להימנע מזיהוי הקו האדום שבלאו הכי הסיכוי יוסף לפונקציה הזאת קטן מאחר שהאינטנסיביות שלו לאחר ההמרה לבינארי קטנה באופן ניכר מהקווים השחורים.

לבסוף אנו אוספים את הקווים עד שמגיעים לחמישה ערכים.

הפונקציה הבאה היא זיהוי הקו האדום. לפונקציה זו אנו מעבירים את התמונה בפורמט RGB שנוצרת לאחר שיחזור התמונה:

```
image = cv2.imread(uploaded_filename)
```

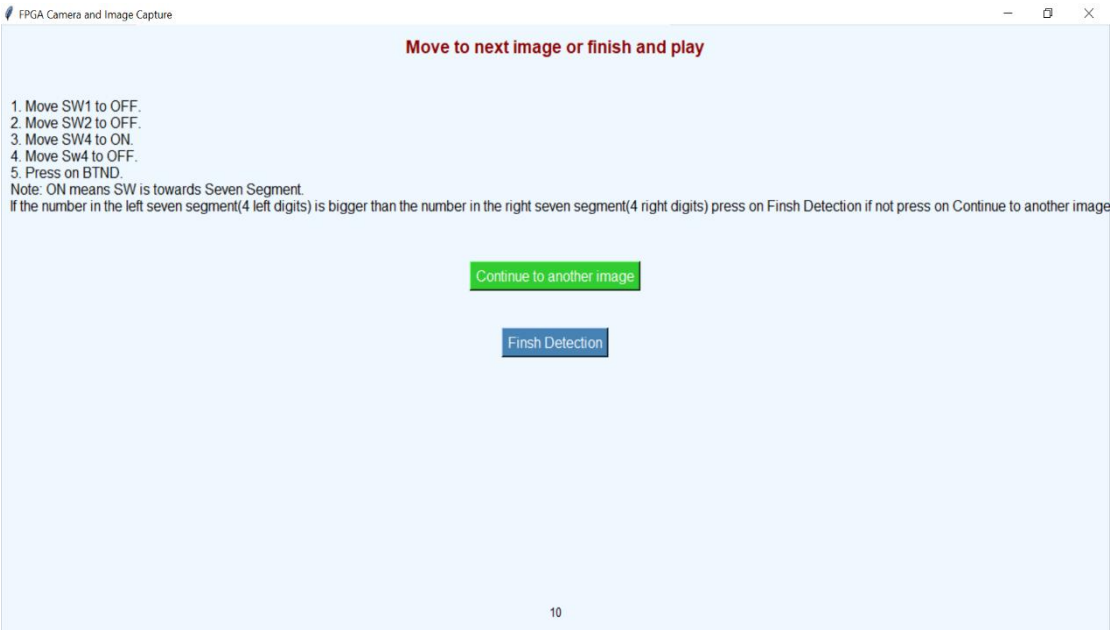
בפונקציה עצמה תחילה אנו משנים את התמונה מ RGB ל HSV, זאת מכיוון שקל יותר לזהות צבעים בפורמט HSV שמייצג באופן ישיר את הצבע (מייצג גוון, רוויה וערך. הצבע יהיה לפי הגוון). הצבע אדום נמצא בערכים שבין 0-10 וגם ב 170-180. בנוסף דרשנו שהרוויה תהיה בין 70-255 והערך 50-255 כך שנבטיח שזה יהיה צבע אדום בולט. לאחר מכן שינינו מחדש את התמונה כל שהאזור האדום נהיה לבן וכל השאר שחור, כך לאחר מכן זיהינו את גובה הקו.

בשלב זה יש לנו משתנה אחד שמכיל את גובה חמשת הקווים השחורים ומשתנה אחר שמכיל את גובה הקו האדום. כעת נותר לנו להשתמש בעוד פונקציה אחת שתבע את התו לפי המיקום של הקו האדום ביחס לקווים השחורים.

בפונקציה זו אנו תחילה מצהירים על מערך שמכיל את כל שמות התווים אותם אנו מצפים לקבל, מהגבוה לנמוך. בכל פעם אנו בודקים אם הקו האדום נמצא מתחת לקו השחור אותו אנו בודקים ולפי כך מתקדמים במערך. אם הקו האדום נמצא בין שני קווים אנו בודקים את המיקום היחסי שלו ולפי כך מחליטים אם הקו צריך להיות ביניהם או שמדובר בסטייה קלה.

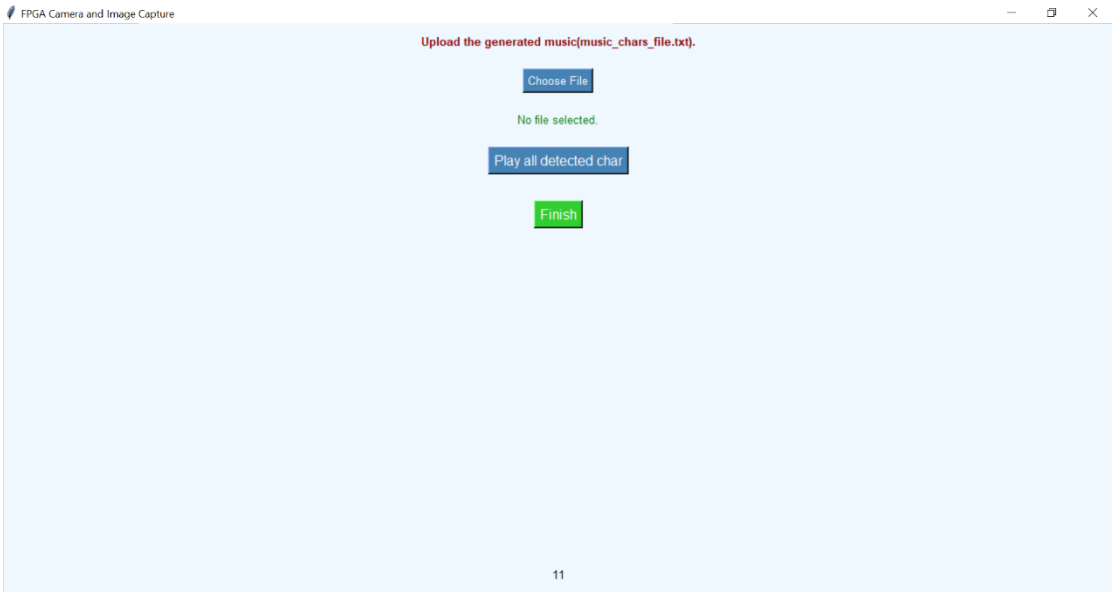
4.2.2.10 **<u>מסך 10- מסך החלטה-</u>** במסך זה מספר הוראות הקשורות לערכת הפיתוח.

מעבר ל-OFF של SW1 ו-SW2 מחזיר אותם לערכים ההתחלתיים שלהם לפני הזיהוי.
עלייה וירידה של SW4 גורם לעלייה של מונה התמונות ב-7 segment ולחיצה על BTND מחזירה את המצלמה למצב וידאו.
לאחר מכן המשתמש אמור לראות תמונת וידאו על מסך התצוגה המחובר ל-VGA. על המשתמש לבחור אם המספר בתצוגת ה-7 segment הימני גדול מהמספר בתצוגת ה- 7 segment השמאלי.
אם לא, יש להמשיך לתמונה הבאה ומתבצעת חזרה למסך 5. אם כן, יש מעבר לחלון הבא.

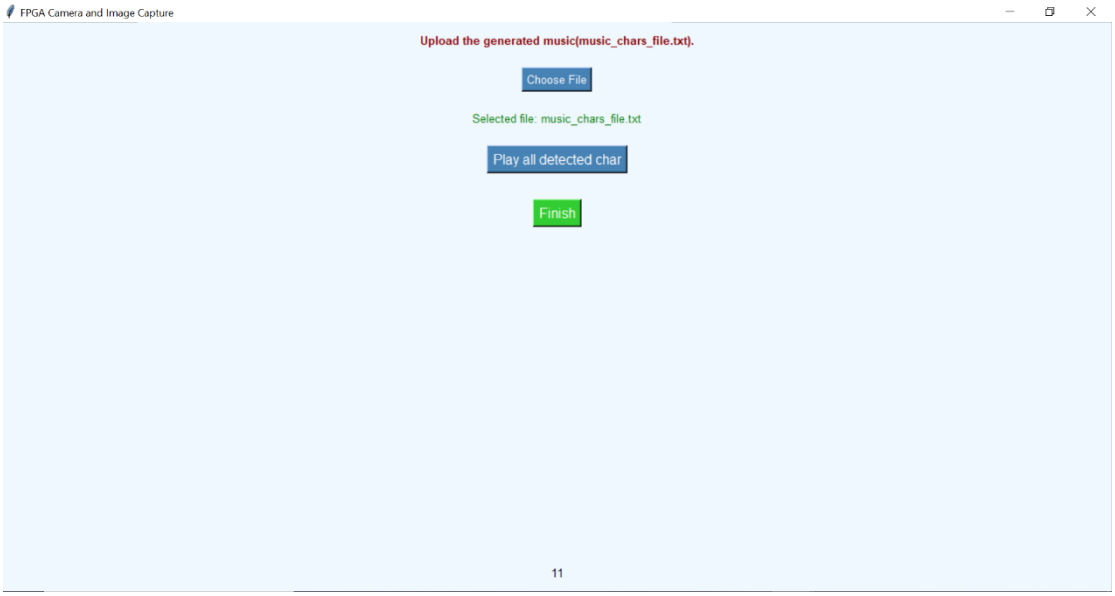


איור 22 : מסך 10 – מסך החלטה

4.2.2.11 **<u>מסך 11 - מסך ניגון התווים-</u>** במסך על המשתמש להעלאות את קובץ הטקסט שמכיל את כל התווים שזוהו. לאחר ההעלאה יש ללחוץ על כפתור Play all detected char.לאחר הלחיצה ה- Arduino יתחיל לנגן את כל התווים שזוהו. לאחר סיום הנגינה יש ללחוץ על לחצן Finish.



איור 23 : מסך 11 – מסך ניגון התווים



איור 24 : מסך 11 – מסך ניגון התווים לאחר העלאת הקובץ

4.2.2.12 **<u>מסך 12 - מסך פרידה-</u>** סיום התהליך וברכת Goodbye על המסך.

איור 25 : מסך 12 – מסך פרידה

4.2.3 Arduino

פלטפורמה מבוססת מיקרובקר שתפקידה בפרויקט שלנו הוא לקבל דרך פרוטוקול UART את נתוני התווים שזוהו ודרך זמזם לנגן אותם. הזמזם מחובר ל-PIN8 ול-GND והוא יודע להפיק תווים על פי התו שנשלח אליו. אנו שולח ב-UART את התו שזוהה אל הארדואינו שם הארדואינו שולח לזמזם את התו שזוהה ואנו שומעים את התווים שזוהו.

בקוד הארדואינו אנו תחילה מעניקים לשמות התווים את התדרים המתאימים באמצעות define

```
#define B3 247
#define C4 262
#define D4 294
```

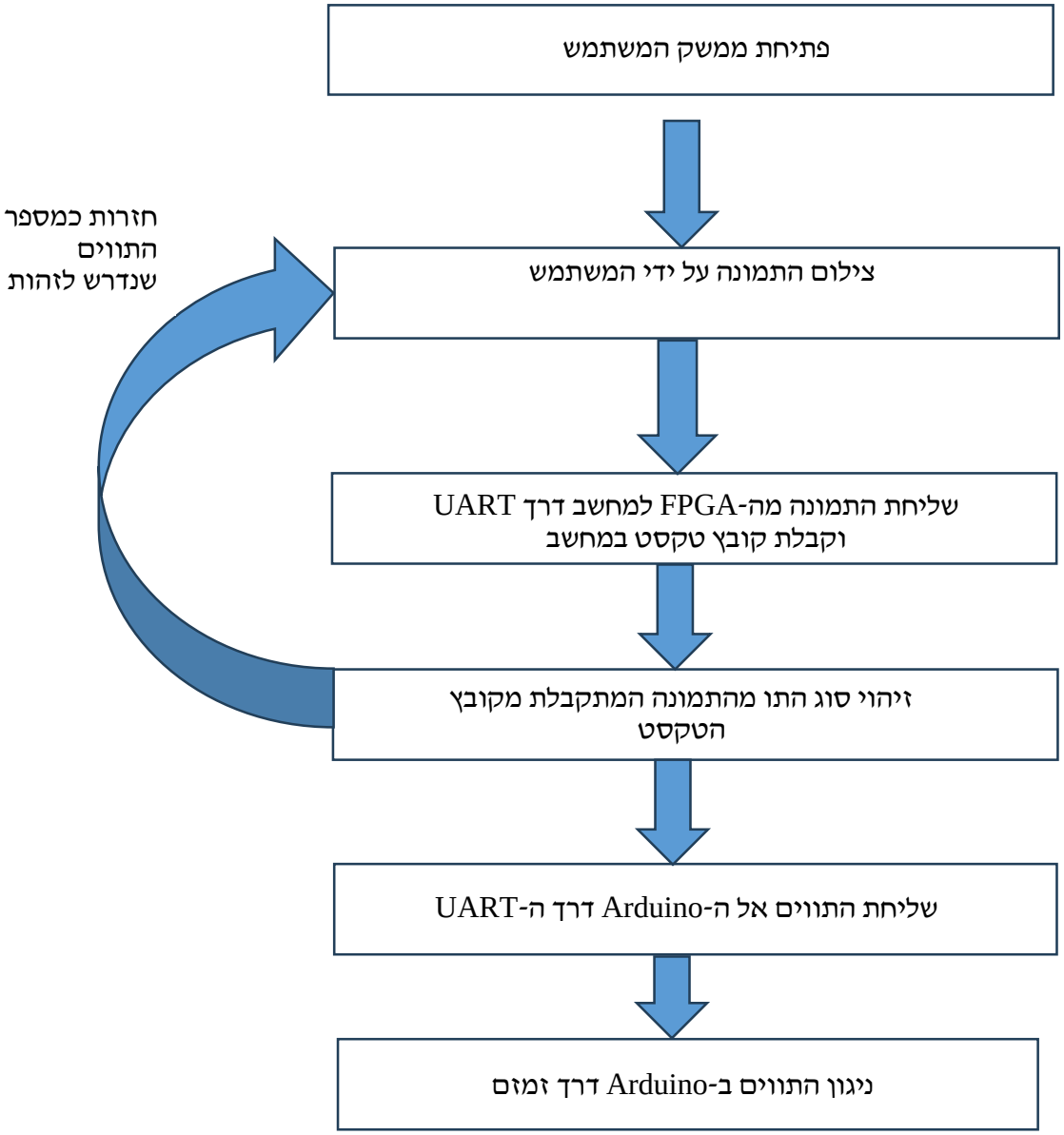
לאחר מכן אנו בודקים שנשלח מידע. במידה וכן אנו שומרים אותו במשתנה data.

```
if (Serial.available() > 0) {
  data = Serial.readStringUntil('\n');  // Read until newline
```

לבסוף אנו מנגנים את התו באמצעות הפונקציה tone. משך התווים מוגדר בקוד כחצי שנייה.

4.2.4 **OV7670** – מודל מצלמה דיגיטלית שמיועד ללכידת תמונות ווידאו ברזולוציה של 480*640. בפרויקט שלנו אנו משתמשים במודל זה כדי לצלם את התו שברצוננו לזהות. המודל מתחבר ל-FPGA דרך קווי data- 8 ביט עבור זרם המידע וממשק SCCB להגדרות המצלמה. בנוסף קיימים קווי סנכרון, שעון ו-reset של המודל המחוברים ל-FPGA. שיקול מרכזי בבחירת מודל זה עבור הפרויקט שלנו היה מחיר יחסית נמוך שימוש נפוץ בפרויקטים אחרים מה שגורם למידע רב באינטרנט על המודל הזה וכמובן מודל ב-git עבור ההגדרה וההפעלה של המצלמה מה שהקל עלינו את עומס העבודה בפרויקט.

4.2.5 **תרשים זרימה של הפרויקט:**



איור 26 : תרשים זרימה של הפרויקט

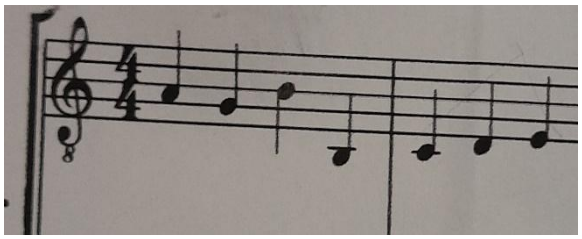
5 תוצאות:

בטבלאות למטה ניתן לראות תיבות שנבדקו ואת התוצאות של הזיהוי תווים.

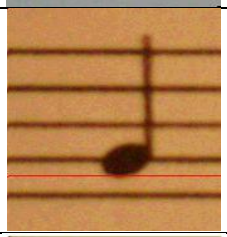
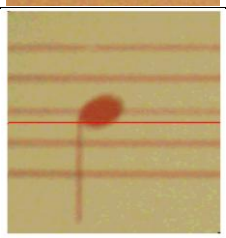
<u>הבהרה</u> : בטבלאות הבאות וכן בכל הפרויקט עצמו השתמשנו בשמות הלועזיים של התווים. כך יוצא ש דו- C, רה- D, מי- E, פה- F, סול- G, לה- A, סי- B. בנוסף לכל תו מוצמד מספר שמסמן את מספר האוקטבה כשהדו האמצעי הוא באוקטבה הרביעית. מיותר לציין שהתדירות של התווים עצמם מגיעה מהשיטה המודרנית בה A4, לה באוקטבה הרביעית, הוא 440Hz.

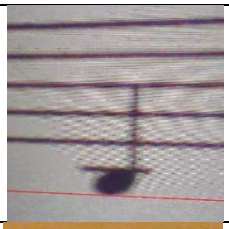
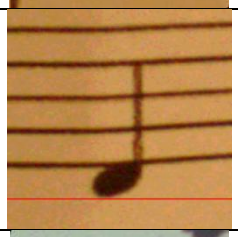
ראשית נתחיל ברצף של מספר תווים שנלקחו לבדיקה ראשונית בנפרד.

<u>תמונה של התיבות</u> :



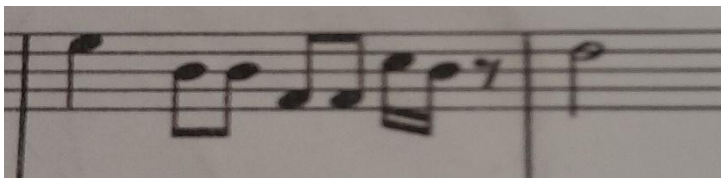
איור 27 : רצף של תווים

התמונה שנלקחה	התו	התו שזוהה	התדר שנוגן
	A4	A4	440Hz
	G4	G4	392Hz
	B4	B4	494Hz

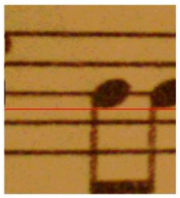
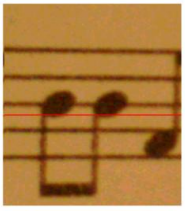
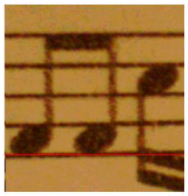
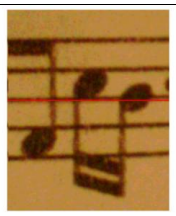
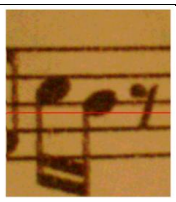
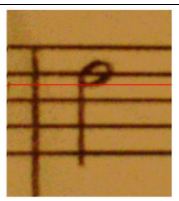
247Hz	B3	B3	
262Hz	C4	C4	
294Hz	D4	D4	
330Hz	E4	E4	

טבלה 2 : תוצאות של מספר דגימות של תמונות של תווים

כעת נתבונן בתיבות הבאות המכילות תווים במשקלים שונים (כאן יש להזכיר שהמערכת אינה מזהה את המשקלים, אלא רק את התווים עצמם).

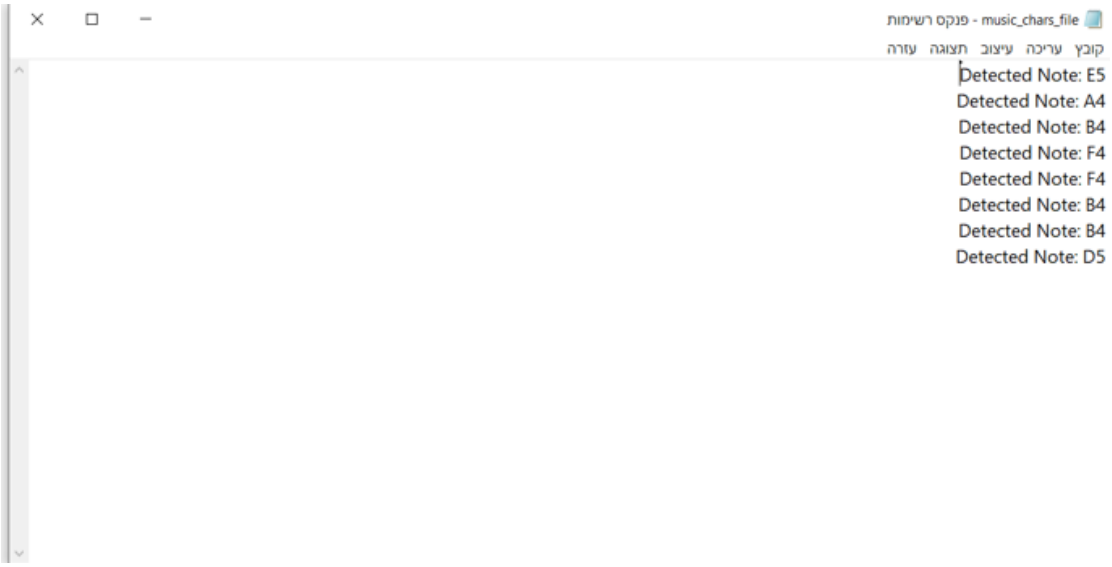


איור 28 : רצף של תווים במשקלים שונים

התמונה שנלקחה	התו	התו שזוהה	התדר שנוגן
	E5	E5	659Hz
	B4	A4	440Hz
	B4	B4	494Hz
	F4	F4	349Hz
	F4	F4	349Hz
	C5	B4	494Hz
	B4	B4	494Hz
	D5	D5	587Hz

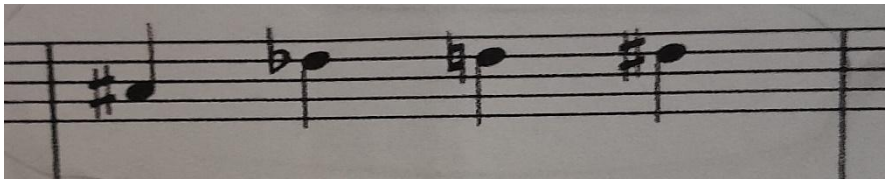
טבלה 3 : תוצאות של רצף תווים עם משקלים שונים

לאחר זיהוי תווים אלו התקבל קובץ טקסט בו רשומים התווים שצריכים להתנגן בסוף הזיהוי.

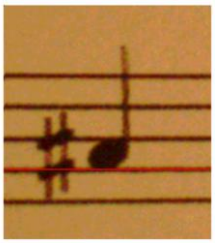
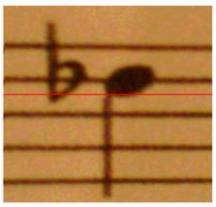
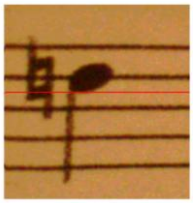
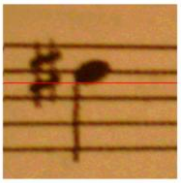


איור 29 : קובץ טקסט של התווים שצריכים להתנגן

כעת נתבונן בתיבה בעלת תווים עם סימנים שונים סימנים שונים (שני תווים עם סימן דיאז, אחד עם במול ועוד אחד עם בקר). גם כאן נזכיר שהמערכת אינה מזהה סימנים ולכן התווים שיתקבלו בסוף יהיו תווים "לבנים" (מבלי סימן).

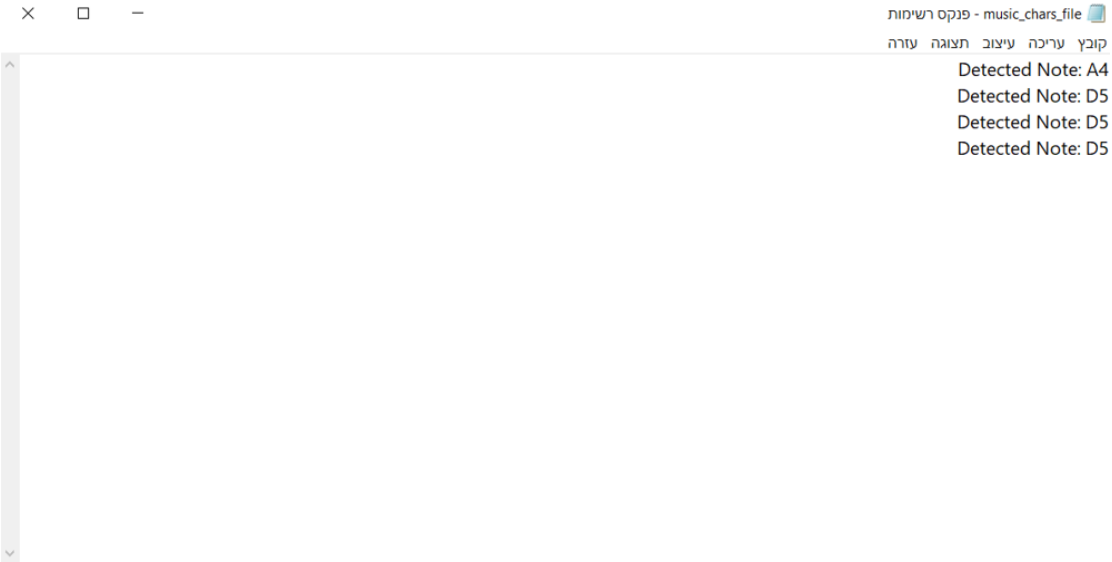


איור 30 : רצף של תווים עם סימנים שונים

התמונה שנלקחה	התו	התו שזוהה	התדר שנוגן
	A#4	A4	440Hz
	Db5	D5	587Hz
	D5	D5	587Hz
	D#5	D5	587Hz

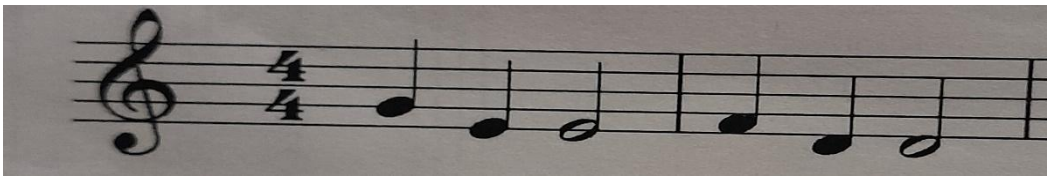
טבלה 4 : תוצאות של רצף תווים עם סימנים שונים

לאחר זיהוי תווים אלו התקבל קובץ טקסט בו רשומים התווים שצריכים להתנגן בסוף הזיהוי.

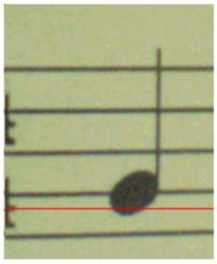
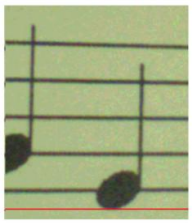
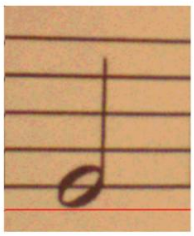
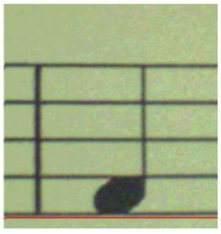
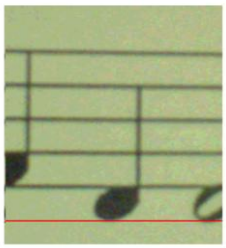
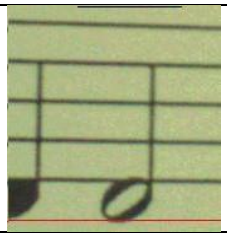


איור 31 : קובץ טקסט של התווים

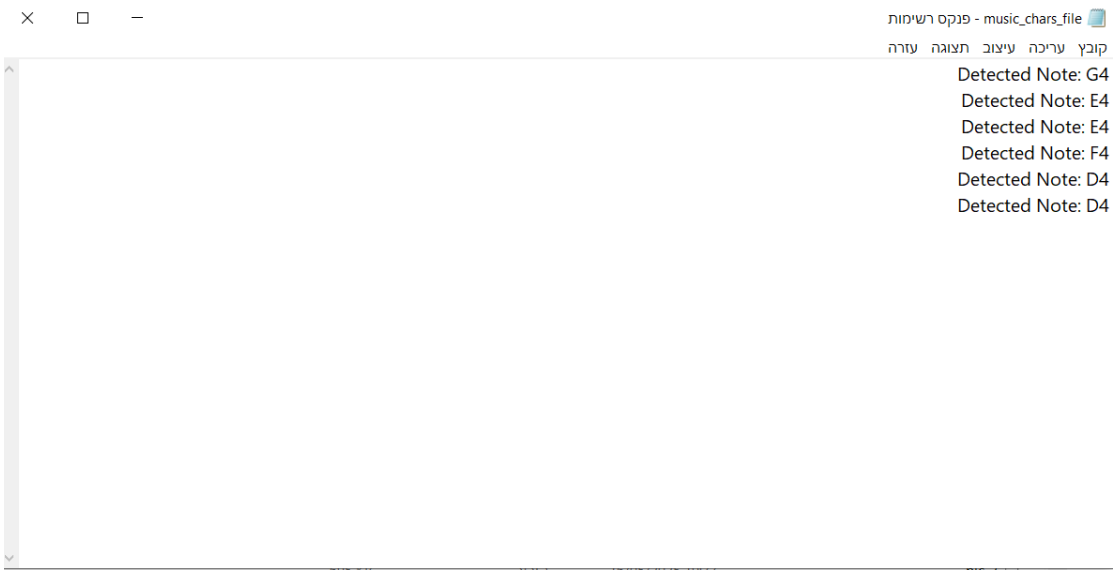
לסיום נבחן את ששת התווים הראשונים של יונתן הקטן.



איור 32 : רצף התווים הראשונים של יונתן הקטן

התמונה שנלקחה	התו	התו שזוהה	התדר שנוגן
	G4	G4	392Hz
	E4	E4	330Hz
	E4	E4	330Hz
	F4	F4	349Hz
	D4	D4	294Hz
	D4	D4	294Hz

טבלה 5 : תוצאות של רצף התווים הראשונים של יונתן הקטן



איור 33 : קובץ טקסט של התווים הראשונים של יונתן הקטן

6 מסקנות והצעות להמשך מחקר:

מהבדיקות שנעשו ניתן להסיק מספר מסקנות. ראשית ניתן לראות שהדיוק בזיהוי התווים הלבנים (חסרי הסימן) הוא כמעט אבסולוטי, את השגיאות המועטות שהיו במקרים האלו סביר להניח שהיה ניתן למנוע על ידי הוספת דיוקים בלוגיקה של הזיהוי תווים, דבר שבהינתן יותר זמן וניסיונות היה ניתן בהחלט. עם זאת בבדיקות מוצגים שני החסרונות העיקריים של שיטה זאת. הראשונה היא זיהוי סימן. התוכנה אינה יודעת לזהות במולים ודיאזים. בעיה זו היא בעיה שקשה מאוד להתגבר עלייה, בעיקר מעצם העובדה שגם אם יש מודל שיודע לזהות את שלושת הסימנים האלה (דיאז, במול ובקר במקרה הצורך) עדיין סימנים אלה יכולים להופיע במקומות שונים על הדף, כלומר ליד התו, בתחילת השורה או ליד תו אחר שבאותה תיבה, ולפי כל אחד מהמקרים האלה צריך לנהוג בהתאם, כלומר יהיה צריך בנוסף לזהות גם התחלה של שורות והתחלה\סיום של תיבות, ועל כך לבנות לוגיקה מתאימה (כלומר להכפיל את התדר בפקטור של כ- 1.059463 במקרים המתאימים). סביר להניח שהדרך הכי פשוטה לבצע פעולה זאת יהיה בעזרת למידה עמוקה, אבל לכך יידרש סט נתונים מתאים.

החיסרון השני שעולה מהתוצאות הוא זיהוי משקל התווים. המודל אינו יודע לזהות את המשקל של התו ומנגן את כולם באותו משך זמן. בשלב זה חשוב לחזור על הנאמר לעיל להזכיר שבגרסה מוקדמת יותר של המודל זיהוי התווים נעשה גם על ידי שימוש בפונקציה של פייתון מחבילת OpenCV שמשמשת לזיהוי אליפסות. שיטה זו הניבה תוצאות טובות עבור תמונות באיכויות גבוהות ותוצאות גבוליות על תמונות שנלקחו ישירות מהערכה שלנו. לבסוף החלטנו לשנות גישה ולהשתמש בשיטה של הקו האדום שמתכוונן לפי המשתמש. שיטה זו נתנה תוצאות הרבה יותר טובות עם דיוק כמעט אבסולוטי עבור התווים הלבנים. עם זאת ייתכן שאם היינו נשארים בגישה הראשונה של זיהוי האליפסה באמצעות ראייה ממוחשבת היה קל לנו יותר לאחר פיתוח המודל להוסיף לו את האפשרות לזהות אם האליפסה חלולה או לאו, ובכך להתחיל לזאת את משקל התו. לאחר מכן להוסיף זיהוי של קו אנכי (שאורכו לא יהיה כל התמונה, אלא מרחק מסוים שיינתן ביחס למרחק הממוצע בין הקווים האופקיים, שמחושב במודל בלאו הכי) ובכך יהיה אפשר לכסות את המקרים של שלם, חצי ורבע. סביר להניח שהמקרים שהם מתחת לרבע יהיו קשיים יותר לזיהוי בגלל הוורסטיליות שלהם והשינויים שקשה יותר לתפוס אותם באמצעות ראייה ממוחשבת (אם כי זה אפשרי). בכל מקרה אנחנו העדפנו לבסוף לשנות גישה כך שמה שיהיה לנו ביד יהיה טוב יותר ולא ללכת בגישה שיהיה לנו דבר פחות טוב ביד אבל עם פוטנציאל עתידי טוב יותר. ככל הנראה אם היינו יכולים להמשיך מכאן את הפרויקט היה עדיף לנו לקחת את הדגימות שכבר יש לנו ולנסות לפתח עליהן את האלגוריתמים הנוספים של הראייה ממוחשבת. גם כאן יש לסייג את הדברים שוב ולהוסיף עוד שתי נקודות חשובות של המסקנות שמכריעות גם בנושא הזה. הראשונה היא שעדיין איכות המצלמה תישאר ירודה, וגם לאחר הפילטרים השונים שאנו משתמשים בקוד (מטשטוש גאוסייאני, לטשטוש מדיין, להעברת התמונות מצורות תצוגה שונות) התמונה יכולה לצאת באיכות שתקשה על אותם אלגוריתמים. הנקודה השנייה שמנעה מאיתנו לעשות מספיק בדיקות במהלך הפיתוח היא משך הזמן שלוקח לשלוח תמונה. כאמור בתחילת הפרויקט משך הזמן היה כשעה, אבל בהמשך הצלחנו להוריד את זמן השליחה משעה לבערך 15 דקות. שינוי זה נעשה בעזרת הקטנת גודל התמונה הנשלחת. עדיין

הדבר הקשה עלינו במהלך הפרויקט לקחת מספיק דגימות ולבנות את האלגוריתמים עליהן (שהרי אלגוריתמים של ראייה ממוחשבת דורשים המון ניסוי וטעייה). משך הזמן של השליחה הוא בעיה שנשארה גם למוצר הסופי. ככל הנראה היה ניתן למנוע זאת על ידי שינוי הבאוד רייט ועל ידי שימוש שונה או באלטרנטיבה מתאימה ל PuTTY, למשל Tera Term או RealTerm שפועלות בצורה מהירה יותר.

לסיכום נראה שהתוצאות עבורן המודל מכוון הן מטיבות ביותר, ויחד עם זאת, התחום של קריאת תווים מוסיקליים הוא עצום ובעל המון אספקטים שונים שרובם אפילו לא נזכרו בכל הכתוב לעיל (למשל תיבות שחוזרות על עצמן, עם שינויים ובלי שינויים, מפתחות שונים כמו מפתח פה ודו וכו') ולכן לבנות מודל שייחסה את כל המקרים הנ"ל זה דבר שאפתני שדורש זמן ומשאבים הרבה מעבר למה שניתן לנו בפרויקט. עם זאת הצלחנו להראות איך בכלים יחסית פשוטים ו"טפשיים" (כלומר ללא בינה מלאכותית) אפשר לבנות מודל פשוט, אינטראקטיבי, שעובד טוב בתחומו ומהווה תשתית טובה לפיתוחים עתידיים.

7 ביבליוגרפיה

[1] Calvo-Zaragoza, J.; Jr, J.H.; Pacha, A. Understanding Optical Music Recognition. *ACM Comput. Surv.* 2020, *53*, 1–35.Pacha* , TU Wien

[2] I. Patel, S. Patel and A. Patel, “Analysis of various image preprocessing techniques for denoising of flower images,” International Journal of Computer Sciences and Engineering, vol. 6, pp. 1111–1117, 2018.

[3] N. Lenssen and D. Needell, "An Introduction to Fourier Analysis with Applications to Music", *Journal of Humanistic Mathematics*, vol. 4, no. 1, pp. 72-91, 2014.